



ITV CyL



PROYECTO INTERMODULAR DE DESARROLLO DE APLICACIONES MULTIPLATAFORMA

**Autores: David Andrés Pérez y Juan Francisco Herreros
González**

Tutoras: Aida Rodríguez Zorrilla y Elena Sarmentero Medina

Curso: 2025-2026

ÍNDICE

1. INTRODUCCIÓN	6
2. FASE DE ANÁLISIS	8
2.1 Descripción de los actores y roles	8
2.2 Especificación de requisitos	8
2.2.1 Requisitos Funcionales	8
2.2.2 Requisitos no funcionales	11
3. FASE DE DISEÑO	13
3.1 Casos de uso	13
3.2 Análisis de persistencia	14
3.2.1 Diagrama Entidad-Relación	17
3.3 Diseño funcional y lógica del sistema	18
3.3.1 Arquitectura del sistema	18
3.3.2 Elección de tecnologías	19
3.3.3 Problemas encontrados y soluciones	19
3.3.4 Scripts desarrollados	20
3.4 Análisis diseño interfaz	26
3.4.1 Elección de JAVA FX y herramientas de desarrollo	26
3.4.2 Conexión con la base de datos	27
3.4.3 Arquitectura	27
3.5 Diseño visual de la interfaz	28
3.5.1 Identidad visual	28
3.5.2 Estructura y navegación	29
4. FASE DE PRUEBAS	32
5. DESPLIEGUE DE LA APLICACIÓN	33
6. DOCUMENTACIÓN DE LA APLICACIÓN	35
7. CONCLUSIONES	36
7.1 Grado de cumplimiento de los objetivos	36
7.1.1 Problemas encontrados	36
7.1.2 Aspectos que han funcionado bien	37
7.2 Trabajo futuro y posibles ampliaciones	37
7.3 Valoración personal	38
8. BIBLIOGRAFÍA	39
Anexo I – Desarrollo de las tareas realizadas	41
Anexo II – Uso de la Inteligencia Artificial en el desarrollo del proyecto	43

1. Herramientas de Inteligencia Artificial utilizadas_____	43
2. Momentos del proyecto en los que se han utilizado _____	43
3. Reporte de Promts _____	44
4. Aportación de la IA al proyecto _____	51
Anexo III – ENLACES DE INTERÉS _____	53

ÍNDICE DE TABLAS

Tabla 1. Requisitos Funcionales	10
Tabla 2. Requisitos No Funcionales	12
Tabla 3. División de Tareas del Trabajo	41

ÍNDICE DE ILUSTRACIONES

Ilustración 1. Diagrama Casos de Uso	15
Ilustración 2. Diagrama Entidad-Relación	17
Ilustración 3. Arquitectura del Sistema	18
Ilustración 4. Ejecución script Descargas.py	21
Ilustración 5. Ejecución script ProcesarTXT.py	21
Ilustración 6. Ejecución script ProcesarCSV.py	24
Ilustración 7. Ejecución script InsertarBBDD.py	25
Ilustración 8. Orden de automatización en PythonAnywhere	26
Ilustración 9. Organización y estructuración del proyecto en JavaFX	28
Ilustración 10. Ventana del Login de la aplicación	29
Ilustración 11. Panel del Administrador	30
Ilustración 12. Ventana de Acondicionamiento Exterior	30
Ilustración 13. Opciones de hash y colisiones esperadas	45
Ilustración 14. Cálculo matemático de colisiones	46
Ilustración 15. Error max_allowed_packet en MySQL	47
Ilustración 16. Script de comprobación de matrículas duplicadas	48
Ilustración 17. Conexión con PythonAnywhere a través del puerto 3306	49
Ilustración 18. Guardado del PDF en la carpeta de descargas del dispositivo	50
Ilustración 19. Ventana emergente de defectos al volver a una pantalla ya visitada	50
Ilustración 20. Ventana emergente de defectos al volver a una pantalla ya visitada	51

1. INTRODUCCIÓN

La Inspección Técnica de Vehículos, conocida popularmente como ITV, es un proceso de revisión periódica y obligatoria en España al que deben someterse todos los vehículos en circulación. Su objetivo es verificar que los vehículos cumplen con los requisitos técnicos y medioambientales exigidos por la normativa vigente, garantizando así la seguridad tanto de sus ocupantes como del resto de usuarios de la vía pública. Las estaciones de ITV son empresas especializadas que realizan estas revisiones y gestionan el registro de sus resultados, lo que implica manejar un volumen considerable de información sobre vehículos, clientes e inspecciones.

El presente Trabajo de Fin de Ciclo recoge el desarrollo de **ITV CyL**, una aplicación de escritorio orientada a la gestión de inspecciones técnicas de vehículos en el ámbito de Castilla y León. Desde el principio teníamos claro que queríamos desarrollar un producto relacionado con vehículos y datos, tomando como punto de partida la base de datos que, de forma anonimizada, publica mensualmente la Dirección General de Tráfico con todos los vehículos matriculados en España. Tras barajar varias ideas (estudio del parque automovilístico español, aplicación de gestión de multas, concesionario con taller integrado), finalmente nos decidimos por una aplicación de ITV por varias razones.

Por un lado, nos permitía trabajar de forma natural con los datos de la DGT, que son precisamente datos de vehículos matriculados. Por otro, la naturaleza de una estación de ITV, cuyos trabajadores operan desde puestos fijos, encaja perfectamente con una aplicación de escritorio, que era el tipo de interfaz que queríamos desarrollar desde el principio. Y por último, el proyecto nos permitía cumplir todos los requisitos académicos del ciclo, especialmente en lo que respecta al trabajo con bases de datos relacionales, que en este caso cobran un papel central, tanto en el *backend* como en el *frontend*.

El proyecto desarrollado supone un ejercicio práctico de integración de tecnologías —*backend* automatizado, base de datos relacional e interfaz gráfica para escritorio— aplicadas a un caso de uso real y cotidiano, centralizando tanto la gestión de los datos del parque vehicular como el registro de las inspecciones realizadas.

La solución desarrollada combina un *backend* automatizado en Python desplegado en PythonAnywhere, que se encarga de descargar, procesar e insertar mensualmente los datos de la DGT en una base de datos MySQL, con una aplicación de escritorio en JavaFX que actúa como interfaz de usuario para los técnicos y administradores de la estación de ITV.

De esta manera, los principales objetivos que se han abordado en el presente trabajo son los siguientes:

- Automatizar la descarga, procesado e inserción mensual de los datos del parque de vehículos de Castilla y León publicados por la DGT.
- Diseñar e implementar un sistema de identificación de vehículos a partir de datos anónimos, generando matrículas únicas y reproducibles mediante funciones de hash criptográfico.

- Desarrollar una base de datos relacional que centralice la información de vehículos, clientes, inspecciones y usuarios del sistema.
- Desarrollar una aplicación de escritorio funcional que permita a los técnicos realizar y registrar inspecciones completas, incluyendo la emisión de un informe en PDF.
- Implementar un sistema de acceso por roles que diferencie las funcionalidades disponibles para técnicos y administradores.

A lo largo de este documento se describirá en detalle las fases de análisis y diseño del sistema, la arquitectura técnica desarrollada, las pruebas realizadas y las conclusiones obtenidas tras el proceso de desarrollo.

2. FASE DE ANÁLISIS

En este apartado se recoge el análisis previo al desarrollo de **ITV CyL**. En primer lugar, se describen los actores que forman parte del sistema y sus roles. A continuación, se especifican los requisitos del proyecto, tanto funcionales como no funcionales, que han servido de base para las decisiones de diseño y desarrollo tomadas a lo largo del trabajo.

2.1 Descripción de los actores y roles

El sistema contempla dos tipos de usuarios, ambos pertenecientes a la empresa que opera la estación de ITV:

- **Técnico:** es el perfil que describe a los empleados encargados de realizar las inspecciones. Su función principal es llevar a cabo las inspecciones técnicas de vehículos, registrar sus resultados y consultar sus propios datos personales. No tiene acceso a la gestión de otros usuarios.
- **Administrador:** empleados que disponen de acceso a la gestión completa de usuarios del sistema. Puede consultar el listado de usuarios registrados y dar de alta, modificar y dar de baja a los miembros del equipo.

Ambos roles requieren autenticación previa para acceder a cualquier funcionalidad de la aplicación.

2.2 Especificación de requisitos

Los requisitos se han establecido a partir del análisis de los casos de uso y de los objetivos del proyecto. Para fijar las prioridades se ha utilizado una escala de tres niveles:

- Alta: para los requisitos imprescindibles, sin los cuales el sistema no puede funcionar correctamente.
- Media: para los requisitos importantes pero que no bloquean el funcionamiento básico.
- Baja: para los requisitos deseables que pueden abordarse en fases posteriores del desarrollo.

2.2.1 Requisitos Funcionales

Los requisitos funcionales describen las capacidades y comportamientos que el sistema debe ofrecer. En la siguiente tabla se pueden encontrar siguiendo el flujo natural del proyecto: primero los relacionados con el *backend* y el procesado de datos de la DGT, y después los relacionados con la aplicación de escritorio.

ID	Nombre	Descripción	Prioridad
RF1	Servidor en la nube	El sistema debe disponer de un servidor remoto en la nube desde el que ejecutar los scripts del <i>backend</i> .	Alta
RF2	Base de datos en la nube	El sistema debe disponer de una base de datos en la nube accesible desde la aplicación de escritorio.	Alta
RF3	Descarga automática de datos DGT	Automatizar la descarga mensual de los archivos ZIP publicados por la DGT en su página oficial, detectando siempre la fecha más reciente disponible.	Alta
RF4	Filtrado por provincias de CyL	El sistema debe extraer únicamente los datos correspondientes a las nueve provincias de Castilla y León.	Alta
RF5	Procesamiento de archivos	El sistema debe procesar los datos extraídos, limpiar los valores incorrectos o vacíos y generar los ficheros de salida con los campos seleccionados.	Alta
RF6	Generación de matrículas únicas	El sistema debe asignar a cada vehículo una matrícula única e irrepetible generada a partir de sus datos técnicos, distinguiendo entre formato moderno y antiguo según el año de primera matriculación.	Alta
RF7	Control de colisiones de matrícula	El sistema debe garantizar que no existen matrículas duplicadas, añadiendo un sufijo numérico en caso de colisión e identificando el registro en la base de datos.	Alta
RF8	Inserción en base de datos	El sistema debe insertar los datos procesados en la base de datos de forma incremental mes a mes, gestionando altas y duplicados.	Alta
RF9	Detección de bajas	El sistema debe detectar los vehículos que ya no aparecen en los datos de la DGT y actualizar su estado en la base de datos.	Alta
RF10	Script coordinador	El sistema debe disponer de un mecanismo que orqueste la ejecución secuencial de todas las etapas del proceso, deteniéndose ante cualquier error.	Alta

RF11	Automatización mensual	El sistema debe ejecutarse de forma automática en las fechas establecidas de cada mes sin intervención manual.	Alta
RF12	Modelo de datos	El sistema debe disponer de las tablas necesarias para gestionar la información de vehículos, usuarios, clientes e inspecciones, con los campos adecuados para cada entidad y las relaciones entre ellas.	Alta
RF13	Autenticación	La aplicación debe mostrar una pantalla de <i>login</i> como primera vista al arrancar y requerir credenciales válidas en todos los casos para acceder al sistema.	Alta
RF14	Gestión de usuarios	La aplicación debe permitir al administrador consultar el listado de usuarios registrados y dar de alta, modificar y dar de baja usuarios del sistema. El técnico no debe tener acceso a esta funcionalidad.	Alta
RF15	Revisión de vehículo	La aplicación debe permitir iniciar el proceso de inspección introduciendo la matrícula del vehículo, recuperando automáticamente sus datos precargados desde la base de datos.	Alta
RF16	Gestión de clientes	La aplicación debe comprobar si existe un cliente asociado a la matrícula al iniciar una inspección y, en caso contrario, permitir crear uno nuevo o cambiar el que había.	Alta
RF17	Registro de inspección	La aplicación debe permitir registrar los resultados de cada fase de la inspección y emitir un informe en PDF al finalizar.	Alta
RF18	Consistencia mensual	El sistema debe garantizar que un mismo vehículo recibe siempre la misma matrícula en cada ejecución del proceso.	Alta
RF19	Gestión de errores	El sistema debe informar de cualquier fallo e interrumpir la ejecución de forma controlada, sin dejar datos incompletos en la base de datos.	Alta

Tabla 1. Requisitos Funcionales

2.2.2 Requisitos no funcionales

Los requisitos no funcionales describen cómo debe comportarse el sistema. Definen criterios de calidad, rendimiento, seguridad, mantenibilidad y otras restricciones que el sistema debe cumplir con independencia de las funcionalidades que ofrece. En la siguiente tabla se recogen los requisitos no funcionales identificados para **ITV CyL**.

ID	Nombre	Descripción	Prioridad
RNF1	Rendimiento	El sistema debe ser capaz de procesar y gestionar aproximadamente dos millones de registros mensuales en un tiempo que no sobrepase los 30 minutos, sin comprometer la estabilidad del servidor.	Alta
RNF2	Seguridad	El acceso a la aplicación debe requerir autenticación obligatoria.	Alta
RNF3	Integridad de los datos	El sistema debe garantizar que los datos no se pierden ni se corrompen. Todas las operaciones sobre la base de datos deben realizarse de forma atómica, deshaciendo los cambios si se produce cualquier error.	Alta
RNF4	Trazabilidad del proceso	El sistema debe informar por pantalla del estado de cada paso del proceso (descarga, extracción, procesado e inserción), facilitando la detección de errores.	Media
RNF5	Modularidad	El código debe estar estructurado en scripts independientes con responsabilidades bien definidas, comentado y documentado para facilitar su mantenimiento y evolución.	Alta
RNF6	Escalabilidad	El sistema debe estar diseñado para poder incorporar nuevas provincias o ampliar el conjunto de datos procesados sin necesidad de reescribir el código.	Media
RNF7	Usabilidad	La interfaz de usuario debe ser clara e intuitiva, permitiendo que los usuarios puedan operar el sistema sin necesidad de formación técnica específica.	Alta
RNF8	Almacenamiento	La base de datos debe ser capaz de almacenar el volumen de datos generado, estimados en torno a 500-600 MB.	Alta

RNF9	Codificación de caracteres	Todos los ficheros generados (CSVs) deben usar codificación UTF-8 con BOM para garantizar la correcta visualización de caracteres especiales como tildes y eñes en cualquier entorno.	Media
RNF10	Diseño IU	La aplicación debe presentar una interfaz visualmente coherente, con una estructura clara y un estilo consistente en todas sus pantallas.	Media
RNF11	Manual de usuario	El proyecto debe incluir documentación de uso dirigida a los usuarios del sistema, que les permita operar la aplicación sin conocimientos técnicos previos.	Media
RN12	Validación de datos de entrada	La aplicación debe validar el formato de los datos introducidos por el usuario antes de procesarlos, rechazando contraseñas que no cumplan los requisitos mínimos de seguridad, teléfonos con formato incorrecto, correos electrónicos inválidos y DNIs con dígito de control erróneo.	Alta

Tabla 2. Requisitos No Funcionales

3. FASE DE DISEÑO

La fase de diseño recoge las decisiones técnicas y arquitectónicas que dan forma a **ITV CyL** y explica cómo se ha planificado su desarrollo antes de pasar a la implementación.

La aplicación tiene como objetivo gestionar el proceso completo de una inspección técnica de vehículos: desde el acceso al sistema mediante autenticación por roles, pasando por la consulta de datos del vehículo y del cliente, el registro de cada fase de la inspección y la emisión del informe final en PDF, hasta la gestión de los usuarios del sistema por parte del administrador. Todo ello respaldado por un *backend* automatizado que mantiene actualizada la base de datos de vehículos con los datos publicados mensualmente por la DGT.

Para llevarlo a cabo, el sistema se articula en torno a tres elementos principales. En primer lugar, los casos de uso definen las interacciones entre los actores del sistema y las funcionalidades que estos pueden realizar según su rol. En segundo lugar, el análisis de persistencia establece cómo y dónde se almacena la información, optando por una base de datos relacional MySQL cuyo modelo de datos se recoge en el diagrama entidad-relación. En tercer lugar, el diseño funcional describe la arquitectura del sistema, tanto el *backend* automatizado en Python como la aplicación de escritorio en *JavaFX*, detallando la lógica de cada componente y las decisiones técnicas tomadas durante el desarrollo. El apartado se completa con los prototipos de la interfaz, que muestran la evolución del diseño visual desde las primeras ideas hasta la propuesta de media fidelidad.

3.1 Casos de uso

Los casos de uso describen las interacciones entre los actores del sistema y las funcionalidades que este les ofrece. En el sistema propuesto intervienen dos actores diferenciados: el técnico y el administrador. Ambos acceden al sistema desde la misma aplicación de escritorio, pero con permisos y flujos de trabajo distintos según su rol.

El técnico es el usuario encargado de realizar las inspecciones. Su flujo comienza siempre con el inicio de sesión, tras el cual accede directamente a la funcionalidad principal: realizar una inspección. El proceso arranca introduciendo la matrícula del vehículo, que el sistema consulta contra la base de datos para recuperar sus datos precargados. En ese momento el sistema también comprueba si existe un cliente asociado a esa matrícula: si lo hay, carga sus datos automáticamente; si no, habilita la opción de añadir un cliente nuevo. A continuación, el técnico recorre las distintas fases de la inspección —documentación, acondicionamiento exterior, chasis y carrocería, acondicionamiento interior, alumbrado y señalización, emisiones contaminantes, frenos, dirección, ejes, ruedas y suspensión, y motor y transmisión— hasta llegar a la pantalla final, donde indica el resultado de la revisión y puede emitir un informe en PDF con todos los datos del vehículo, el cliente, los fallos detectados y las observaciones registradas a lo largo del proceso.

El administrador, tras iniciar sesión, accede a la gestión de usuarios del sistema. Desde una pantalla con el listado completo de usuarios registrados puede dar de alta nuevos técnicos o

administradores, modificar los datos de cualquier usuario existente y dar de baja a quienes ya no formen parte del equipo.

3.2 Análisis de persistencia

Para el almacenamiento de los datos del sistema hemos optado por una base de datos relacional MySQL alojada en *PythonAnywhere*, la misma plataforma que utilizamos para ejecutar los scripts del *backend*. Esta decisión responde a varias razones.

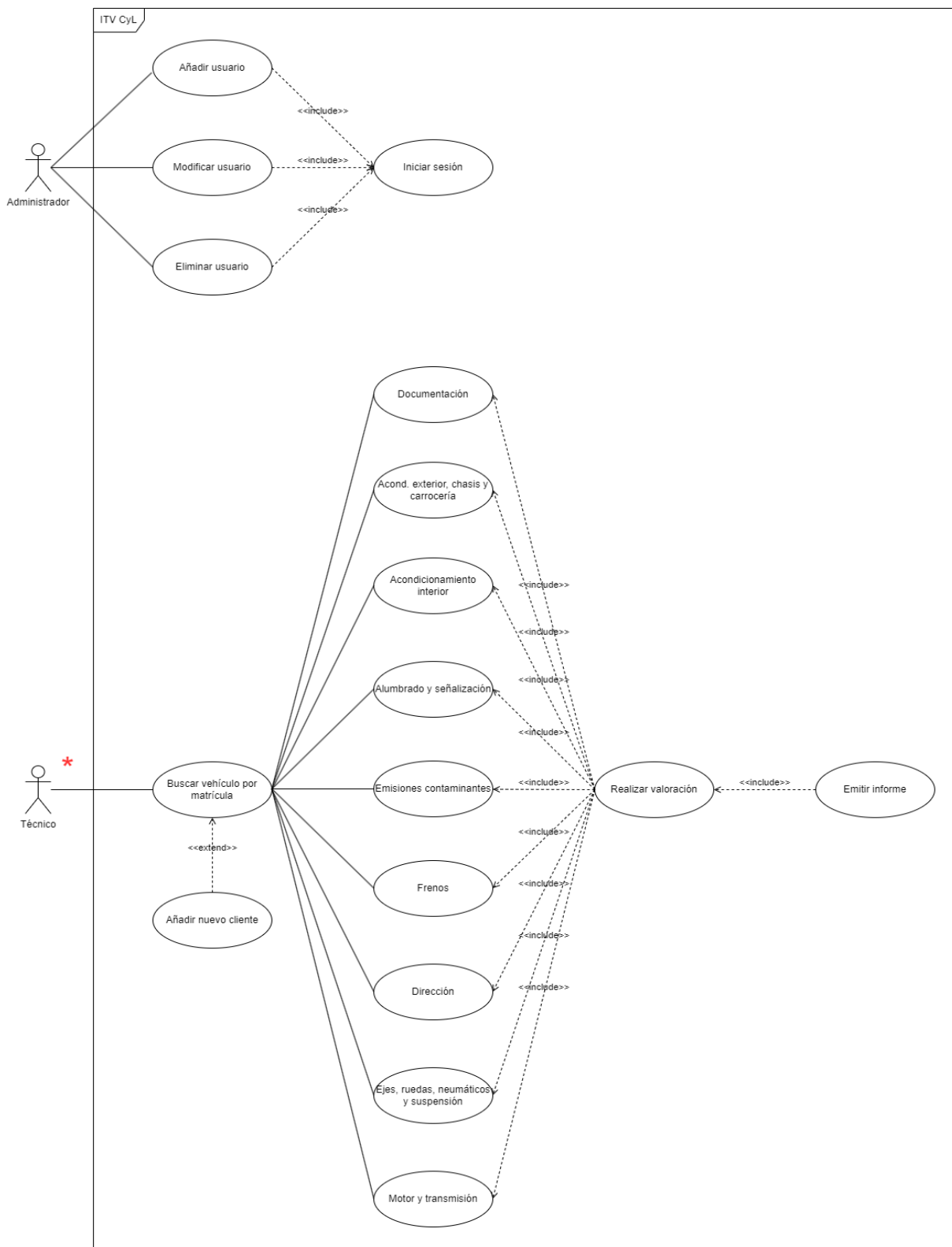
La primera es la integración natural con el resto del sistema. Al tener el *backend* y la base de datos en la misma plataforma, la comunicación entre los scripts de Python y MySQL es directa y eficiente, sin necesidad de configurar conexiones externas ni gestionar infraestructuras adicionales.

La segunda es que MySQL fue desarrollada para manejar datos relacionales. Los vehículos se asocian a clientes, los clientes a inspecciones, las inspecciones a sus defectos y los usuarios al sistema de acceso. Este tipo de relaciones entre entidades se modela de forma natural y robusta en una base de datos relacional, con claves foráneas que garantizan la integridad referencial.

La tercera es la fiabilidad y el soporte de MySQL. Es el sistema gestor de bases de datos relacionales más extendido en entornos profesionales, con una documentación amplia y una compatibilidad total tanto con Python (a través de MySQL -connector- python) como con Java (a través de JDBC), los dos lenguajes de programación utilizados en el desarrollo.

El modelo de datos se compone de siete tablas:

- **VEHICULOS_CYL**: almacena los datos del parque de vehículos procesados mensualmente desde los ficheros de la DGT, con la matrícula generada como clave primaria.
- **CLIENTES**: recoge los datos de los clientes que acuden a la estación de ITV. El campo DNI tiene restricción de unicidad para evitar duplicados en el sistema.
- **VEHICULO_CLIENTE**: actúa como tabla intermedia para modelar la relación N:M entre vehículos y clientes, ya que un mismo vehículo puede ser llevado a inspeccionar por distintas personas a lo largo del tiempo.
- **INSPECCIONES**: registra cada revisión técnica realizada, asociada a la matrícula del vehículo correspondiente mediante clave foránea.
- **INSPECCION_DEFECTOS**: almacena los defectos concretos detectados en cada inspección, vinculados a la inspección que los originó. Se elimina en cascada si se elimina la inspección.



* Para los casos de uso del técnico, también es necesario 'Iniciar sesión', que se relacionaría con un include con dichos casos. No lo dejamos representado gráficamente debido a que creemos que afectaría negativamente a la lectura del diagrama

Ilustración 1. Diagrama Casos de Uso

- **USUARIOS:** gestiona el acceso al sistema, almacenando las credenciales y el rol de cada miembro del equipo. Tanto el nombre de usuario como el correo electrónico tienen restricción de unicidad, garantizando que no pueden existir dos usuarios con los mismos datos de acceso. No tiene relación con el resto de las tablas porque en el contexto de esta aplicación no se ha considerado almacenar en Base de Datos qué técnico ha realizado cada inspección.
- **TARIFAS:** almacena los precios de la inspección según el tipo de vehículo. Es una tabla independiente, sin relaciones con el resto del modelo, porque las tarifas no están ligadas a ninguna inspección concreta ni a ningún vehículo en particular: son datos de referencia que controlamos desde el código de *JavaFX*.

3.2.1 Diagrama Entidad-Relación

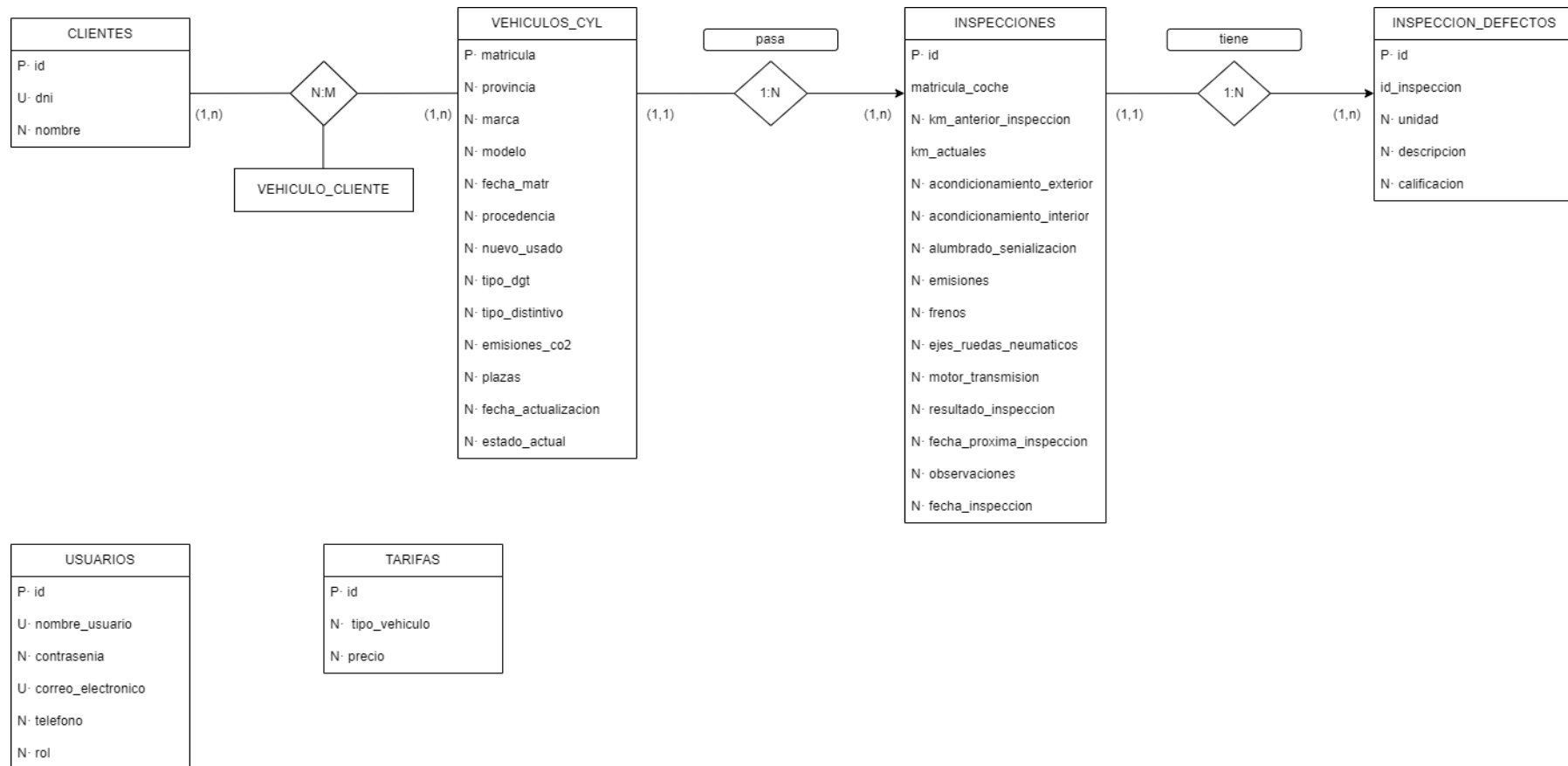


Ilustración 2. Diagrama Entidad-Relación

3.3 Diseño funcional y lógica del sistema

En este apartado se describen las decisiones técnicas y arquitectónicas que han dado forma al sistema. Se explica cómo está estructurado, qué tecnologías hemos utilizado y por qué, qué problemas encontramos durante el desarrollo y cómo los resolvimos, y cómo está organizado el código en scripts con responsabilidades bien definidas.

3.3.1 Arquitectura del sistema

El sistema se estructura en tres capas bien diferenciadas que trabajan de forma coordinada: el *backend* automatizado, la base de datos y la aplicación de escritorio.

El *backend*, alojado en *PythonAnywhere*, es el encargado de acceder, manipular, filtrar e insertar en base de datos los registros de la DGT. Existen cinco scripts, de los cuales los cuatro primeros forman una cadena de procesamiento: *descarga.py* obtiene los datos de la DGT, *procesarTXT.py* los filtra y los extrae, *procesarCSV.py* los transforma y genera las matrículas, e *insertarBBDD.py* los sube a la base de datos. El quinto script, *coordinador.py*, actúa como punto de entrada único que orquesta a los otros cuatro y gestiona la automatización mensual.

La base de datos MySQL, alojada en *PythonAnywhere*, almacena la información de los vehículos, los usuarios del sistema, los clientes y las inspecciones realizadas, actuando como punto de unión entre el *backend* y el *frontend*.

La aplicación de escritorio, desarrollada en *JavaFX*, es la herramienta que usan los trabajadores en su día a día. Se conecta a la base de datos para consultar los datos de los vehículos, registrar inspecciones y gestionar los usuarios del sistema.

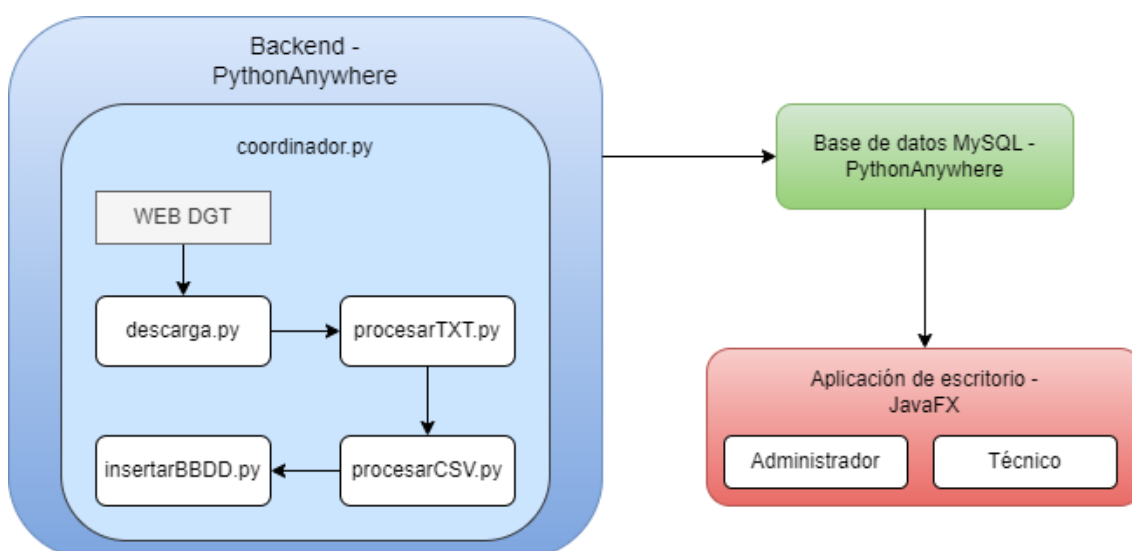


Ilustración 3. Arquitectura del Sistema

3.3.2 Elección de tecnologías

La elección de Python para el *backend* fue una decisión clara desde el principio. Frente a Java, que era la otra opción que se manejaba por ser el lenguaje principal del ciclo, Python ofrece una experiencia de trabajo mucho más ágil cuando se trata de procesar y transformar datos: librerías como *pandas*, *requests* o *BeautifulSoup* permiten resolver en pocas líneas tareas que en Java requerirían un desarrollo considerablemente más extenso. Dado que el núcleo del *backend* es precisamente el procesamiento de grandes volúmenes de datos, Python era la elección natural.

En cuanto a la plataforma de alojamiento, se optó por *PythonAnywhere* debido a su integración nativa con Python y a la facilidad con la que permite desplegar scripts, configurar tareas programadas y gestionar bases de datos MySQL desde un único entorno. De esta manera se puede tener el *backend* funcionando en producción de forma sencilla y sin necesidad de gestionar infraestructura propia.

Para la base de datos se ha elegido MySQL principalmente por su compatibilidad con *PythonAnywhere*, que lo ofrece de forma integrada en sus planes, y por ser el sistema gestor de bases de datos relacional más extendido en entornos profesionales. Su uso con Java desde el *frontend* mediante JDBC es también bien conocido y está ampliamente documentado.

3.3.3 Problemas encontrados y soluciones

El desarrollo del sistema no estuvo exento de dificultades. A continuación, se describen los principales problemas que se presentaron y cómo fueron resueltos, aunque cada uno de ellos se desarrolla con más detalle en la sección correspondiente de cada script.

El primer problema fue la anonimización de los datos. Los ficheros de la DGT no contienen ningún identificador único por vehículo —solo características técnicas— por lo que no había forma directa de distinguir dos coches idénticos ni de asignarles un identificador estable. Lo se resolvió mediante un sistema de doble hash: el primero registra la combinación de características técnicas y el segundo, combinado con un número de instancia, identifica a cada vehículo de forma individual y perdurable.

El segundo problema fue el volumen de datos. Manejar cerca de dos millones de registros mensuales impone restricciones tanto en memoria como en almacenamiento. Se resolvió extrayendo del ZIP de la DGT únicamente los ficheros de las provincias que nos interesan, en lugar de descomprimirlo completo, y eliminando el ZIP una vez terminada la extracción para liberar espacio en disco.

El tercer problema fue el rechazo de la DGT a las conexiones realizadas para la descarga automática de datos. La versión gratuita de *PythonAnywhere* enruta todas las conexiones a través de un proxy compartido, y la página de la DGT bloqueaba las peticiones que provenían de ese proxy, impidiendo la descarga automática de los datos. Dicho problema se resolvió migrando al plan de pago de la plataforma, que permite conexiones directas a internet sin restricciones de proxy.

Relacionado con lo anterior, la versión gratuita también imponía un límite de 512 MB para la base de datos, insuficiente para el volumen de datos que se manejan. El plan de pago amplió

ese límite a 5 GB, lo que aportó espacio más que suficiente para las necesidades actuales del proyecto.

Por último, durante el desarrollo se detectó que la provincia de Ávila no aparece de forma consistente en los ficheros publicados por la DGT, que trata de forma irregular los nombres de provincia que contienen tildes. Se resolvió este problema aplicando una normalización de caracteres antes de comparar los nombres de los ficheros, de forma que el sistema pueda procesar Ávila cuando aparece y continuar sin ella cuando no lo hace, sin necesidad de modificar el código.

3.3.4 Scripts desarrollados

El *backend*, como hemos mencionado anteriormente, está compuesto por cinco scripts de Python, cada uno con una responsabilidad única y bien definida. Esta arquitectura responde al esquema de diseño modular, que establece que un sistema debe dividirse en componentes independientes con funciones claramente delimitadas. Aplicar este principio facilita el mantenimiento, ya que cualquier fallo es inmediatamente localizable en el script responsable; la evolución del sistema, ya que modificar una parte del proceso no afecta al resto; y la reutilización, ya que un componente concreto podría integrarse en otro sistema sin necesidad de arrastrar el resto del pipeline.

Descarga.py

El primer script del pipeline tiene una única responsabilidad: detectar y descargar el archivo ZIP más reciente publicado por la DGT con los datos del parque de vehículos mensual.

Para llevar a cabo su función hace uso de cuatro librerías: *os* para trabajar con rutas y carpetas del sistema, *re* para buscar patrones de fecha dentro del HTML de la DGT mediante expresiones regulares, *requests* para realizar la descarga tanto de la página web como del archivo ZIP, y *BeautifulSoup* para interpretar el HTML y extraer el texto visible de forma sencilla.

Al inicio del script se centralizan en constantes la carpeta de destino de los ZIPs, la dirección oficial de la página de la DGT y el patrón de la URL del archivo ZIP, con un marcador de fecha que se sustituye automáticamente en cada ejecución.

El núcleo del script es la función que detecta la fecha más reciente publicada por la DGT. En lugar de que alguien la introduzca manualmente, el script descarga la página oficial, extrae su texto con *BeautifulSoup* y aplica una expresión regular para encontrar todas las cadenas con formato “YYYYMM”. Con ese formato, la comparación alfabética coincide exactamente con la comparación cronológica, por lo que basta con quedarse con el valor máximo para obtener siempre la fecha más reciente. Con esa fecha construye la URL exacta del ZIP y lo descarga en modo binario, byte a byte, sin ninguna transformación de codificación, ya que se trata de un archivo comprimido y no de texto plano. Como en el resto de *scripts*, toda la lógica se centraliza en una función *main()* que ejecuta los pasos en el orden correcto.

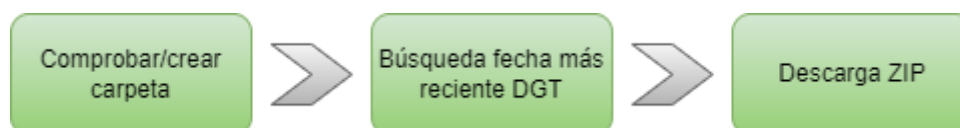


Ilustración 4. Ejecución script *Descargas.py*

ProcesarTXT.py

El segundo script del pipeline abre el ZIP descargado, extrae únicamente los ficheros TXT de las provincias de Castilla y León y elimina el ZIP una vez terminado el proceso para liberar espacio en disco.

Para trabajar con archivos comprimidos utiliza la librería `zipfile`. Además, importa la función de detección de fecha de *descarga.py*, lo que le permite saber qué ZIP debe procesar sin que sea necesario indicarlo manualmente.

El núcleo del script es la función que recorre todos los ficheros contenidos en el ZIP y extrae solo los que corresponden a las provincias escogidas. Para que la comparación sea robusta independientemente de cómo haya nombrado la DGT los ficheros ese mes, aplica una normalización de caracteres con `unicodedata` antes de comparar, eliminando tildes y convirtiendo todo a mayúsculas. Esto resuelve el problema de que “Ávila”, “León” o cualquier otra provincia con tilde pueda aparecer con variantes distintas según el mes.

Para evitar extraer el mismo fichero más de una vez, el script mantiene un conjunto de nombres ya procesados que consulta antes de cada extracción. Una vez terminado, comprueba si el ZIP sigue existiendo en disco y lo elimina, conservando únicamente los TXT necesarios para el siguiente paso.

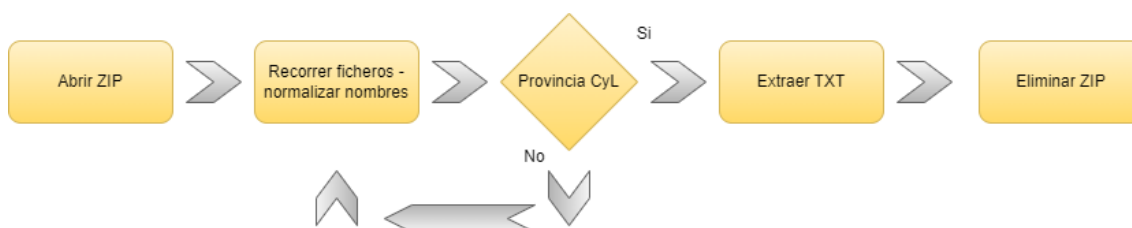


Ilustración 5. Ejecución script *ProcesarTXT.py*

ProcesarCSV.py

El tercer script del pipeline es el más complejo del proyecto y el que ha concentrado la mayor parte del trabajo técnico. Su función es leer los ficheros TXT extraídos, procesar sus datos y generar nueve archivos CSV, uno por cada provincia, listos para ser insertados en la base de datos. La decisión de generar un fichero por provincia en lugar de uno único con todos los datos responde a una limitación práctica que detectamos durante el desarrollo: al intentar trabajar con un único CSV que agrupara todos los registros, la carga era excesiva para Excel, que terminaba dando error. Dividir los datos por provincia nos permite, además, localizar y gestionar

los errores de forma mucho más controlada, sabiendo exactamente en qué provincia se ha producido el problema.

El primer desafío que se planteó al comienzo de este script es uno de naturaleza estructural: los ficheros de la DGT no contienen ningún identificador único por vehículo. No hay ninguna clave primaria, ningún número de bastidor, ningún dato que permita distinguir un registro de otro de forma directa. Los datos son completamente anónimos desde el punto de vista identificativo. Para desarrollar el sistema era imprescindible crear ese identificador, con la condición de que fuera estable —es decir, que el mismo vehículo produjera siempre el mismo identificador en cada ejecución mensual— y que fuera significativo para los usuarios del sistema.

Dado que los datos corresponden a vehículos, lo más razonable era que el identificador tuviera formato de matrícula. Este formato es reconocible e intuitivo para cualquier persona que trabaje en una estación de ITV —y para cualquier persona en general—, es fácil de introducir en la aplicación y permite al técnico asociar mentalmente el identificador del sistema con el vehículo que tiene delante. Una solución técnicamente equivalente como un código numérico aleatorio habría funcionado igual, pero habría resultado confusa, anti intuitiva y difícil de manejar en el contexto real de uso.

Para construir ese identificador partimos de los propios datos técnicos del vehículo que nos proporciona la DGT. De todos los campos disponibles, seleccionamos aquellos que describen las características físicas e inmutables del vehículo: datos relacionados con su construcción, mecánica, dimensiones y homologación. Excluimos deliberadamente todos los campos que reflejan situaciones administrativas o circunstanciales, como el tipo de titular, el número de propietarios o la fecha de matriculación actual, porque esos datos pueden cambiar a lo largo de la vida del vehículo sin que el vehículo en sí haya cambiado. El criterio de selección fue por tanto claro: solo los datos del vehículo que son inmutables.

Con esos campos construimos el sistema de generación de matrículas, cuyo funcionamiento se basa en dos niveles de hash¹. El algoritmo elegido es MD5, que convierte cualquier cadena de texto en una secuencia de 32 caracteres hexadecimales. MD5 está considerado obsoleto para aplicaciones de seguridad, ya que es vulnerable a ciertos tipos de ataques criptográficos, pero en este caso no lo usamos con fines de seguridad sino como función de dispersión determinista: lo que nos interesa es que la misma entrada produzca siempre la misma salida, que sea rápido y que genere una distribución suficientemente uniforme de resultados. Para ese propósito, MD5 es completamente adecuado y, además, su bajo coste computacional es beneficioso cuando hay que aplicarlo a millones de registros.

El primer hash se calcula a partir de los datos técnicos seleccionados del vehículo. Este hash identifica el tipo de vehículo. Así, por ejemplo, dos coches con exactamente las mismas características técnicas producirán el mismo primer hash. El problema surge precisamente ahí, porque en una flota de casi dos millones de vehículos es habitual que existan miles de unidades

¹ Un hash es una función matemática que transforma cualquier conjunto de datos (texto, archivos, contraseñas) en una serie alfanumérica única de longitud fija, actuando como una "huella digital" inmutable.

con características idénticas —mismo modelo, misma motorización, mismo año, misma provincia—, y todos ellos generarían el mismo hash y, por tanto, la misma matrícula.

Para resolverlo introducimos el concepto de instancia. Llevamos un contador por cada hash: el primer vehículo con unas características determinadas es la instancia 0, el segundo es la instancia 1, el tercero la instancia 2, y así sucesivamente. El segundo hash se calcula combinando el primer hash con el número de instancia, lo que garantiza que nunca dos vehículos pueden compartir el mismo segundo hash. De esta manera, dos vehículos con las mismas características producen el mismo primer hash pero, al tener distinto número de instancia, generan segundos hashes y matrículas (en teoría) completamente diferentes.

Aun así, existe una pequeña probabilidad matemática de que dos vehículos distintos generen la misma combinación de dígitos y letras al convertir el hash en matrícula. Esto ocurre porque estamos comprimiendo un hash de 128 bits en un espacio de combinaciones posibles mucho más reducido, y con casi dos millones de registros las colisiones son estadísticamente inevitables según la paradoja del cumpleaños². Cuando se produce una colisión —controladas en código un mapa global de matrículas ya asignadas—, se añade un sufijo numérico a la matrícula generada y la marcamos con el estado “*Descartada*”. Este sufijo hará que se eviten problemas estructurales irresolubles a este nivel en la base de datos. De esta manera se preserva la integridad de los datos: el vehículo existe en los ficheros de la DGT y debe estar representado en la base de datos. Lo que indicamos con ese estado es que su identificador no sigue el formato limpio de una matrícula real y que, por tanto, no es un registro primario del sistema. Los valores posibles del campo ESTADO_ACTUAL son A para “*Activa*”, D para “*Descartada*” y B para “*Baja*”.

En cuanto a las matrículas generadas, siguen dos formatos: para los vehículos matriculados desde el año 2000 se ha usado el formato actual de cuatro dígitos y tres letras; para los anteriores, el formato antiguo en el que las dos primeras letras identifican la provincia. Los dos formatos usan alfabetos distintos que reflejan la realidad histórica de las matrículas españolas: las modernas usan un alfabeto de 21 consonantes, excluyendo la Ñ y las vocales para evitar combinaciones que puedan resultar inapropiadas, mientras que las antiguas usan un alfabeto

² Para entender por qué ocurre, basta con comparar el número de vehículos que manejamos con el número de combinaciones posibles. Las matrículas modernas, con cuatro dígitos y tres consonantes, ofrecen un espacio de 92.610.000 combinaciones posibles; las antiguas, con las letras de provincia fijas y dos letras del alfabeto completo de 25 letras (excluyendo la ‘Ñ’), suman aproximadamente 50.000.000 combinaciones adicionales repartidas entre las ocho provincias. Cabe recordar que todos estos cálculos se realizan sobre los datos de ocho provincias de Castilla y León, ya que Ávila no aparece de forma consistente en los archivos de la DGT. Con 1.875.073 registros procesados y aplicando la paradoja del cumpleaños — que establece que la probabilidad de colisión crece de forma cuadrática con el número de elementos — el modelo predice unas 15.750 colisiones, lo que representaría aproximadamente un 0,84% del total. Los datos reales obtenidos arrojan 14.069 matrículas descartadas sobre un total de 1.875.073 registros, lo que supone un 0,75% del total, con 1.861.004 matrículas activas. Un resultado que se ajusta con relativa precisión a la predicción teórica.

de 25 letras que sí incluye las vocales, siendo más fiel al formato que se usó en España antes del año 2000³.

Un problema detectado durante el desarrollo fue que la función `apply()` de `pandas`, usada inicialmente para recorrer las filas de forma eficiente, no garantiza el orden estricto de ejecución cuando hay efectos secundarios en variables globales como el mapa de instancias. Esto provocaba matrículas duplicadas sin sufijo. La solución fue hacer uso del bucle `iterrows()`, que aunque es algo más lento garantiza que el procesado es siempre secuencial y predecible.

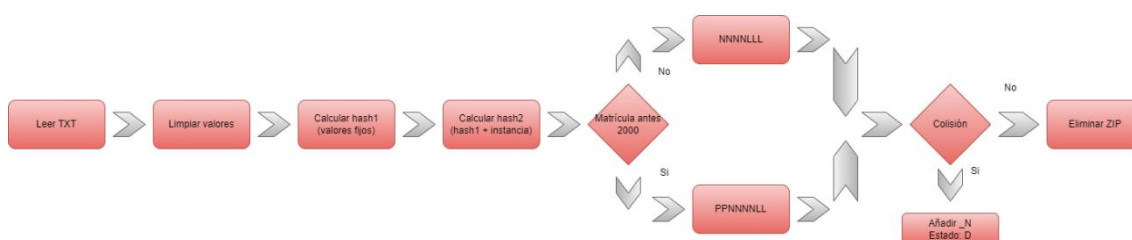


Ilustración 6. Ejecución script *ProcesarCSV.py*

Antes de calcular ningún hash, cada valor pasa por una función de limpieza con tres filtros en cascada que normalizan los datos incorrectos, vacíos o con caracteres no imprimibles a un valor neutro, garantizando que dos vehículos con las mismas características siempre produzcan el mismo hash. Una vez generadas las matrículas, el script aplica conversiones de legibilidad sobre los campos codificados — provincia, procedencia y nuevo/usado — y añade la fecha y hora de ejecución como campo de auditoría. Los CSVs se guardan con codificación UTF-8 con BOM⁴ para que Excel los abra correctamente.

InsertarBBDD.py

El cuarto script lee los nueve CSVs generados e inserta su contenido en la base de datos MySQL de forma incremental y segura mes a mes. Utiliza la librería `MySQL -connector -python` para conectarse a MySQL.

Antes de procesar ningún CSV, carga en memoria todas las matrículas existentes en la base de datos mediante una consulta `SELECT`, almacenándolas en un conjunto de Python para poder hacer búsquedas instantáneas independientemente del volumen. Para cada fila de cada CSV, el script compara la matrícula con ese conjunto: si ya existe, la salta; si no, la inserta. Usa `INSERT`

³ Debido a su casi nulo valor estadístico, hemos decidido no implementar el formato provincial numérico vigente en España de 1900 a 1971, incluyendo esos casos en el modelo de matrícula provincial alfanumérico.

⁴ UTF-8 con BOM (*Byte Order Mark*) es una variante de la codificación UTF-8 que incluye una marca de tres bytes al inicio del fichero. Esta marca indica al programa que lo abre que el contenido está codificado en UTF-8, lo que permite a Excel interpretar correctamente caracteres especiales como tildes y eñes. Sin esa marca, Excel abre el fichero asumiendo la codificación del sistema operativo, lo que provoca que dichos caracteres aparezcan distorsionados.

IGNORE como medida de seguridad adicional para que cualquier duplicado inesperado sea descartado silenciosamente sin abortar el proceso.

Al mismo tiempo acumula todas las matrículas de los CSVs en un segundo conjunto. Al terminar, la resta entre el conjunto de matrículas de la base de datos y el de los CSVs del mes actual identifica los vehículos que ya no aparecen en los datos de la DGT, que se marcan como “Baja” mediante UPDATE. Para evitar superar el límite de tamaño de paquete de MySQL, las bajas no se procesan en una única consulta sino en lotes de 1.000 matrículas, lo que garantiza que la operación funciona correctamente independientemente del volumen de bajas detectadas.

Todas las operaciones se realizan dentro de una transacción: si el proceso termina sin errores se confirman con `commit`; si falla en cualquier punto se deshacen con `rollback`, garantizando que la base de datos nunca queda en un estado inconsistente. La conexión se cierra siempre al final gracias al bloque `finally`.

Un aspecto que requirió atención especial fue la conversión de fechas: los CSVs usan el formato español “DD/MM/YYYY” mientras que MySQL espera “YYYY-MM-DD” para los campos de tipo DATE y con hora para los de tipo DATETIME. Se implementó una función de conversión que acepta ambos formatos y devuelve el valor esperado, o nulo si el valor es inválido.

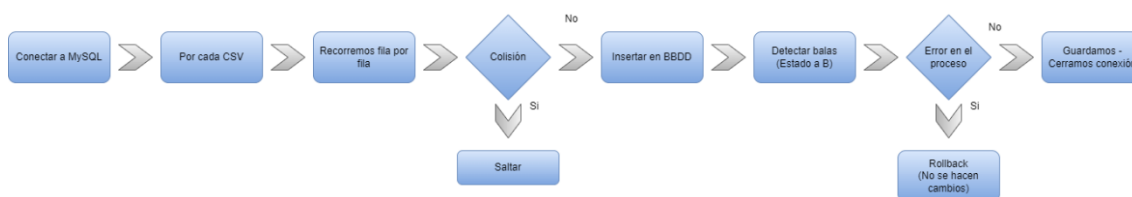


Ilustración 7. Ejecución script *InsertarBBDD.py*

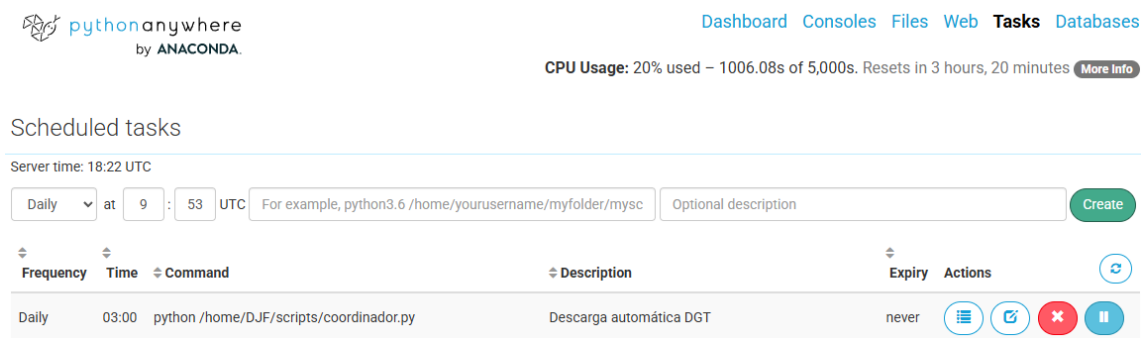
Coordinador.py

El último script del proyecto no procesa ningún dato por sí mismo: importa y ejecuta en orden los cuatro scripts anteriores, convirtiendo todo el pipeline en un proceso que puede lanzarse con un único comando.

Antes de arrancar el pipeline, el coordinador ejecuta un paso previo de limpieza que elimina por completo la carpeta temporal donde guardamos el ZIP, los TXT y los CSVs. Aunque en teoría, al insertar los datos de un nuevo mes estos se sobrescribían sobre los anteriores, no nos hemos querido arriesgar y preferimos que cada ejecución parta siempre desde cero, sin ningún fichero residual que pueda interferir en el procesado.

Cada script se ejecuta dentro de una función que envuelve la llamada en un bloque de control de errores. Si el script completa su ejecución correctamente el coordinador pasa al siguiente; si falla, muestra el nombre del paso y el motivo del error y detiene el proceso de inmediato. Los cuatro scripts son un todo indivisible — no tiene sentido ejecutar uno sin los anteriores — por lo que ante cualquier fallo lo correcto es parar y notificar.

Para la automatización mensual el coordinador se configura como tarea diaria en *PythonAnywhere* a las tres de la madrugada. Sin embargo, al inicio de cada ejecución comprueba si el día actual es el 5 o el 20 del mes — las fechas en las que la DGT suele haber publicado los datos nuevos — y solo actúa si lo es.



The screenshot shows the PythonAnywhere dashboard. At the top, there's a navigation bar with links: Dashboard, Consoles, Files, Web, Tasks, and Databases. Below this, a status bar indicates CPU Usage: 20% used - 1006.08s of 5,000s. Resets in 3 hours, 20 minutes. The main section is titled 'Scheduled tasks'. It shows a form to create a new task with a dropdown for frequency (set to 'Daily'), a time field (set to '9:53 UTC'), a command field (containing 'python3.6 /home/yourusername/myfolder/mysc'), and an optional description field. A 'Create' button is on the right. Below the form is a table of existing tasks:

Frequency	Time	Command	Description	Expiry	Actions
Daily	03:00	python /home/DJF/scripts/coordinador.py	Descarga automática DGT	never	[List] [Refresh] [Delete] [Pause]

Ilustración 8. Orden de automatización en *PythonAnywhere*

3.4 Análisis diseño interfaz

Este apartado recoge las decisiones de diseño tomadas para la interfaz de usuario de **ITV CyL**, la justificación de las tecnologías elegidas y la arquitectura seguida en el desarrollo del *frontend*.

3.4.1 Elección de *JAVAFX* y herramientas de desarrollo

Para el desarrollo de la interfaz de usuario se eligió *JavaFX* como tecnología principal. Durante el ciclo formativo se ha trabajado con Apache NetBeans para crear aplicaciones de escritorio, por lo que se ha orientado esta aplicación hacia *JavaFX* como una alternativa más moderna y potente que mantiene una curva de aprendizaje razonable para quien ya conoce el entorno de escritorio en Java. Esta recomendación ha permitido desarrollar una interfaz más cuidada y moderna que la que se habría conseguido con las herramientas que ya se conocían.

Para el diseño visual de las pantallas se ha utilizado *Scene Builder*, una herramienta que permite construir interfaces gráficas de forma visual mediante arrastrar y soltar componentes (*drag and drop*), generando automáticamente los ficheros FXML que luego se integran en el proyecto. Para la parte funcional, el código Java que da vida a cada pantalla, se ha usado Visual Studio Code como entorno de desarrollo, con la extensión de Java y las librerías de *JavaFX* configuradas manualmente.

Todas las pantallas de la aplicación parten de un panel base sobre el que se combinan distintos tipos de contenedores según las necesidades de cada vista. Cada pantalla tiene una imagen de fondo asignada mediante un componente *ImageView*, lo que le da a la aplicación una apariencia coherente y profesional. Los estilos visuales se aplican mediante propiedades CSS con el prefijo `-fx`. Cada componente tiene asignado un identificador mediante el atributo `fx:id`, que permite referenciarlo desde el código Java del controlador correspondiente, y cada ventana tiene asociada una clase controladora mediante la propiedad *Controller* de *Scene Builder*.

3.4.2 Conexión con la base de datos

La conexión entre la aplicación *JavaFX* y la base de datos MySQL alojada en *PythonAnywhere* presentó una dificultad técnica inicial: *PythonAnywhere* no permite conexiones directas al puerto estándar de MySQL (3306) desde el exterior por razones de seguridad. Para resolverlo se han seguido las instrucciones oficiales de la plataforma, que explican cómo establecer la conexión de forma correcta desde una aplicación externa. La lógica de conexión está centralizada en un único fichero dentro del paquete **CONNECTION** del proyecto, lo que facilita cambiar las credenciales o el *host* sin tener que modificar el resto del código.

3.4.3 Arquitectura

Para estructurar el proyecto se ha seguido el patrón de diseño DAO (*Data Access Object*), que es una de las buenas prácticas más extendidas en el desarrollo de aplicaciones con acceso a base de datos. Este patrón separa de forma clara la lógica de acceso a datos del resto de la aplicación, lo que facilita el mantenimiento, las pruebas y la evolución del código. Si en el futuro se decide cambiar la base de datos o modificar alguna consulta, los cambios quedan aislados en la capa DAO sin afectar al resto del sistema.

El proyecto se organiza en los siguientes paquetes dentro de la carpeta *src*:

- **CONNECTION** contiene la clase encargada de establecer y gestionar la conexión con la base de datos de *PythonAnywhere*.
- **DAOS** agrupa las clases DAO del proyecto, una por cada entidad del sistema, que encapsulan todas las operaciones de consulta, inserción, modificación y eliminación sobre la base de datos.
- **POJOS** contiene las clases POJO (*Plain Old Java Object*), que son clases simples que representan las entidades del sistema — vehículo, cliente, inspección, usuario — con sus atributos y métodos *getter* y *setter* correspondientes.
- **CONTROLLERS** incluye los controladores de cada ventana, que son las clases Java que gestionan la lógica de cada pantalla: responden a las acciones del usuario, llaman a los DAOs para obtener o guardar datos y actualizan la vista en consecuencia.
- **UI** contiene las clases correspondientes a las ventanas de la aplicación.
- **vistas** almacena los ficheros FXML generados con *Scene Builder*, que definen la estructura visual de cada pantalla.
- **img** es la carpeta donde se guardan todas las imágenes utilizadas en la aplicación, de forma que sean accesibles cuando el proyecto se exporta.



Ilustración 9. Organización y estructuración del proyecto en JavaFX

Para gestionar el flujo de datos durante una inspección, que implica recorrer varias pantallas sucesivas acumulando información, se ha implementado una clase de utilidad que actúa como buffer temporal. Esta clase almacena los datos que el técnico va introduciendo en cada pantalla de la inspección para que estén disponibles al final del proceso, tanto para guardarlos en la base de datos como para generar el informe PDF.

Para la generación del informe PDF se ha utilizado la librería Thymeleaf, que permite definir una plantilla HTML con variables que se sustituyen en tiempo de ejecución por los datos reales de la inspección, y OpenHTMLtoPDF, que convierte ese HTML a PDF aplicando el CSS definido en la plantilla.

Además de estas, el proyecto incluye otras clases de utilidad para gestionar la sesión del usuario autenticado, controlar las opciones del menú lateral de la inspección y centralizar la lógica de apertura y cierre de ventanas.

3.5 Diseño visual de la interfaz

El diseño de la interfaz de ITV CyL ha evolucionado a lo largo del desarrollo, desde los primeros prototipos de baja fidelidad hasta la versión final de la aplicación. En ese proceso se han tomado decisiones conscientes sobre colores, tipografía, iconos, estructura y navegación que responden tanto a criterios estéticos como funcionales.

3.5.1 Identidad visual

La paleta de colores de la aplicación se inspira en los colores heráldicos de Castilla y León: el rojo grana y el dorado. Estos colores aparecen en los elementos de la interfaz que pretendíamos resaltar —cabeceras, botones principales y marcadores de sección— dotando a la aplicación de una identidad visual reconocible y vinculada al ámbito territorial en el que opera.

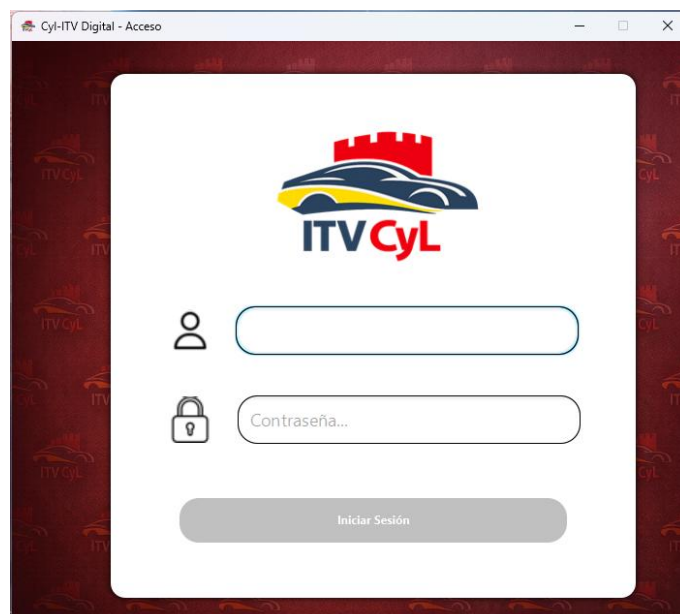


Ilustración 10. Ventana del Login de la aplicación

Sin embargo, durante el desarrollo y tras revisar aplicaciones reales de ITV, se ha detectado que las interfaces más efectivas en entornos profesionales de uso intensivo son aquellas que minimizan la carga visual. Por ese motivo, la versión final abandonó el fondo dorado de los prototipos en favor de un fondo blanco como base, complementado con grises para elementos secundarios como el menú lateral o los separadores de sección. El dorado corporativo se reserva para los acentos y puntos de atención, de forma que guía la mirada del usuario sin saturar la pantalla.

Para la tipografía se optó por la fuente del sistema que proporciona *JavaFX* por defecto, que responde al mismo perfil que fuentes como Arial o Calibri: limpia, sin remates y de alta legibilidad en pantalla. Esta elección permitía además no introducir dependencias tipográficas externas cuando la fuente del sistema ya cumplía exactamente con lo que se buscaba.

Los iconos utilizados son iconos estándar de *FlatIcons*, lo que garantiza su escalabilidad y coherencia visual en todas las pantallas.

3.5.2 Estructura y navegación

Como se ha visto a lo largo del informe, la aplicación presenta dos flujos claramente diferenciados según el rol del usuario autenticado.

El flujo del administrador sigue una estructura clásica de gestión: una pantalla principal con el listado completo de usuarios y pantallas secundarias para dar de alta, modificar o eliminar. Estas pantallas secundarias, aunque muestran datos distintos, comparten la misma estructura de formulario, lo que reduce la curva de aprendizaje del usuario.



Ilustración 11. Panel del Administrador

El flujo del técnico es más elaborado y se articula en torno al proceso de inspección. La primera pantalla es singular: muestra dos paneles en paralelo, uno con los datos del vehículo recuperados automáticamente de la base de datos y otro con el historial de inspecciones anteriores, lo que permite al técnico contextualizar la revisión antes de comenzar. A partir de ahí, las pantallas centrales de la inspección siguen un patrón homogéneo: cada una corresponde a una subsección técnica (frenos, alumbrado, emisiones...) y presenta los mismos elementos — encabezado, elementos a valorar junto a un desplegable para indicar su estado y, si se detecta un error, un modal para describir el defecto. Esta uniformidad permite al técnico operar con rapidez una vez que ha interiorizado el patrón, sin necesidad de reaprender la interfaz en cada paso.



Ilustración 12. Ventana de Acondicionamiento Exterior

La última pantalla también es singular: es donde el técnico indica el resultado global de la inspección y genera el informe en PDF. Al generar dicho informe, el sistema calcula automáticamente la tarifa correspondiente a la inspección consultando la tabla de tarifas de la base de datos y comparándola con el tipo de vehículo y su distintivo ambiental. Este cálculo queda reflejado en el PDF junto a los datos del cliente, los datos del vehículo, los fallos encontrados y el resultado final.

En cuanto a la navegación, los prototipos iniciales planteaban un flujo completamente lineal mediante botones de siguiente. En la versión final se mantiene esa tendencia natural de avanzar paso a paso, pero se incorpora un menú lateral que permite saltar a cualquier sección de la inspección en cualquier momento. Esto responde a una necesidad real del proceso: si un técnico necesita corregir o completar un apartado anterior, no debe estar obligado a recorrer toda la secuencia de nuevo. El menú lateral actúa como índice visual de la inspección, ofreciendo además una referencia constante de en qué punto del proceso se encuentra el usuario.

La pantalla de emisiones contaminantes incorpora un indicador que muestra automáticamente los valores máximos permitidos para el vehículo que se está inspeccionando, calculados en función de su distintivo ambiental y su año de matriculación. De esta forma el técnico dispone en todo momento de la referencia necesaria para valorar los resultados sin tener que consultarla externamente. En los casos en que el vehículo no tiene valores de referencia aplicables, el indicador muestra N/A.

Así mismo, la aplicación incorpora un conjunto de validaciones en los formularios de entrada de datos para garantizar la integridad de la información registrada. Las contraseñas deben cumplir unos requisitos mínimos de seguridad: al menos ocho caracteres, una letra mayúscula, una minúscula, un número y un carácter especial. Los números de teléfono deben tener nueve dígitos y empezar por 6, 7, 8 o 9, en consonancia con la numeración telefónica española. Los correos electrónicos deben seguir el patrón estándar de <dirección@dominio.extensión>. Y los DNI introducidos al registrar un nuevo cliente son verificados comprobando que el dígito de control es el correcto para los números introducidos, siguiendo el algoritmo oficial de validación del documento nacional de identidad español.

En el Anexo III está disponibles los enlaces para acceder a los prototipos originales, tanto el de Baja Fidelidad como el de Media Fidelidad.

4. FASE DE PRUEBAS

Una vez completado el desarrollo del sistema, es necesario verificar que tanto el *backend* como la aplicación de escritorio se comportan según lo esperado y cumplen con los requisitos definidos en la fase de análisis. Para ello, se ha llevado a cabo una fase de pruebas estructurada que cubre los dos componentes del proyecto de forma independiente.

En el caso del *backend*, las pruebas se centran en verificar que el pipeline de procesamiento de datos funciona correctamente de principio a fin: que la descarga detecta siempre la versión más reciente de los datos de la DGT, que la extracción filtra correctamente las provincias de Castilla y León, que la generación de matrículas es única y reproducible, que la inserción en base de datos gestiona correctamente altas, duplicados y bajas, y que el coordinador orquesta todo el proceso de forma segura y predecible.

En el caso del *frontend*, las pruebas verifican el comportamiento de la aplicación desde el punto de vista del usuario: que la autenticación y el control de acceso por roles funcionan correctamente, que el administrador puede gestionar usuarios sin errores, que el técnico puede realizar el ciclo completo de una inspección, y que las validaciones de los formularios rechazan correctamente los datos incorrectos o incompletos.

Las pruebas se han documentado en un plan de pruebas que recoge 54 casos organizados por módulo, con sus precondiciones, acciones, resultado esperado y resultado obtenido. Cada caso tiene asignada una prioridad y un estado de ejecución, lo que permite hacer un seguimiento claro del grado de cobertura alcanzado. Igualmente, se han seguido las recomendaciones del SSTQB en cuanto a utilizar evidencias comunes en varios casos de prueba, evitar pruebas repetitivas que no ayudan a la calidad y la utilidad de flujos completos.

[Enlace al plan de pruebas](#)

5. DESPLIEGUE DE LA APLICACIÓN

El despliegue del proyecto abarca dos componentes con naturalezas muy distintas: el *backend*, que ya se ejecuta de forma remota en *PythonAnywhere* y no requiere instalación en el equipo del usuario, y la aplicación de escritorio *JavaFX*, que debe poder instalarse en cualquier equipo Windows de la estación de ITV de forma sencilla y sin que el usuario necesite conocimientos técnicos.

Para la distribución del *frontend* se ha generado un instalador ejecutable para Windows utilizando dos herramientas: *jpakege* y *WiX Toolset*.

- *jpakege* es una utilidad que forma parte del propio JDK de Java desde la versión 14 y permite empaquetar una aplicación Java junto con su entorno de ejecución en un instalador nativo.
- *WiX Toolset* es la herramienta que *jpakege* utiliza internamente en Windows para construir el fichero ejecutable de instalación.

La razón principal de usar este enfoque es que el instalador resultante incluye el entorno de ejecución de Java dentro del propio paquete, lo que significa que el equipo donde se instale la aplicación no necesita tener Java instalado previamente. Esto es especialmente relevante en el contexto de una estación de ITV, donde no ha de asumirse que los equipos dispongan de ningún entorno de desarrollo configurado.

WiX Toolset requiere una instalación previa en el sistema. Para ello se descarga su ejecutable, se instala y se añade su ruta a las variables de entorno del sistema, concretamente a la variable *Path*, de forma que *jpakege* pueda invocarlo desde cualquier terminal sin necesidad de indicar la ruta completa cada vez. Para comprobar que la instalación se ha realizado correctamente basta con ejecutar el comando `candle -?` en una terminal, que devuelve la lista de opciones disponibles de la herramienta.

Antes de generar el instalador fue necesario realizar una serie de preparativos en el proyecto. En primer lugar, se creó en Visual Studio Code una clase auxiliar denominada *Lanzador*, cuya única función es invocar la pantalla de *login* al arrancar la aplicación. Este intermediario es necesario porque *jpakege* requiere que la clase principal no extienda directamente de *Application*, que es la clase base de *JavaFX*, y la clase de *login* lo hacía. Una vez creada esta clase, se tiene que actualizar el fichero *launch.json* del proyecto para que apunte a *Lanzador* como clase principal en lugar de a la clase de *login* original.

A continuación, se exporta el fichero JAR de la aplicación desde Visual Studio Code mediante el comando `Java: Export Jar`, seleccionando *Lanzador* como clase principal y excluyendo las librerías externas del JAR, ya que estas se añadirán por separado. Con el JAR generado, se tiene que preparar la carpeta de entrada del instalador, que debe contener el JAR principal, todos los ficheros JAR de las librerías externas que hay en la carpeta *lib* del proyecto y los ficheros DLL necesarios para que *JavaFX* funcione correctamente en Windows.

Para el icono de la aplicación se convirtió el logotipo del proyecto de formato PNG a formato ICO mediante una herramienta de conversión online, ya que Windows requiere el formato ICO

para los iconos de las aplicaciones. Este fichero se coloca en la raíz de la carpeta del proyecto, fuera de la carpeta de entrada.

Con todo preparado, el instalador se genera ejecutando el comando de *jpackage* desde una terminal de Visual Studio Code apuntando al JDK instalado en el equipo. El comando especifica el tipo de salida como ejecutable .exe, la carpeta de destino donde se guardará el instalador, el nombre de la aplicación, su versión, la carpeta de entrada con todos los recursos, el JAR principal, la clase de entrada, el icono y el nombre del autor. Se añaden además tres opciones orientadas a mejorar la experiencia de instalación: la creación automática de un acceso directo en el escritorio, la incorporación de la aplicación al menú de inicio de Windows y la posibilidad de que el usuario elija la carpeta de instalación. Por último, se incluye la opción que fuerza a *JavaFX* a usar renderizado por software, lo que evita errores de aceleración gráfica en equipos que no disponen de tarjeta gráfica dedicada, que es una situación habitual en equipos de trabajo de oficina.

El resultado es un fichero ejecutable que cualquier usuario puede lanzar para instalar la aplicación en su equipo de forma guiada, sin necesidad de configurar ningún entorno de desarrollo ni instalar Java ni ninguna dependencia adicional.

6. DOCUMENTACIÓN DE LA APLICACIÓN

Con el objetivo de facilitar tanto la puesta en marcha de la aplicación como su uso cotidiano, se han elaborado dos documentos de apoyo dirigidos a perfiles distintos.

El manual de instalación está orientado al administrador del sistema o al personal técnico autorizado, y describe paso a paso el proceso de instalación de ITV CyL en un equipo Windows, incluyendo los requisitos previos necesarios y el flujo completo del asistente de instalación.

El manual de usuario está dirigido a todos los usuarios de la aplicación — tanto técnicos como administradores — y explica el funcionamiento de cada funcionalidad disponible según el rol, desde el acceso al sistema hasta la generación del informe PDF al finalizar una inspección.

Ambos documentos están disponibles en los siguientes enlaces:

- [Manual de instalación](#)
- [Manual de uso](#)

7. CONCLUSIONES

7.1 Grado de cumplimiento de los objetivos

Al inicio del proyecto definimos cinco objetivos principales que guiarían el desarrollo de ITV CyL. A la vista del resultado obtenido, podemos afirmar que todos ellos se han cumplido satisfactoriamente.

El primero era diseñar e implementar un sistema de identificación de vehículos a partir de datos anónimos. Este objetivo ha sido el más exigente del proyecto y el que más trabajo técnico ha concentrado, pero el resultado es un sistema de doble hash que genera matrículas únicas, reproducibles y con un formato coherente con la realidad de las matrículas españolas.

El segundo era automatizar la descarga, procesado e inserción mensual de los datos del parque de vehículos de Castilla y León. El pipeline funciona de forma completamente desatendida: de forma automática los días programados el sistema descarga, procesa e inserta en la base de datos cerca de dos millones de registros sin intervención manual.

El tercero era desarrollar una base de datos relacional que centralizara la información del sistema. La base de datos está operativa en *PythonAnywhere* con las seis tablas necesarias, las relaciones correctamente modeladas y los datos de prueba cargados para las verificaciones.

El cuarto era desarrollar una aplicación de escritorio funcional. La aplicación permite al técnico introducir una matrícula y recuperar automáticamente todos los datos del vehículo desde la base de datos, completar el proceso de inspección y generar un informe en PDF con un acabado profesional que reproduce fielmente el estilo de los informes reales de ITV.

El quinto y último era implementar un sistema de acceso por roles. El sistema distingue correctamente entre técnico y administrador, mostrando a cada uno únicamente las funcionalidades que le corresponden.

-

7.1.1 Problemas encontrados

El problema más significativo del proyecto fue, sin duda, el de la identificación de los vehículos. Los ficheros de la DGT no contienen ningún identificador único por vehículo, lo que obligó a diseñar desde cero un sistema propio. La solución del doble hash con control de instancias no fue inmediata: requirió varias iteraciones, el uso de herramientas de comprobación de duplicados y un análisis matemático de las colisiones esperadas para validar que el modelo era correcto. El resultado final es robusto y reproducible, pero llegar a él supuso el mayor esfuerzo técnico del proyecto.

El segundo problema relevante fue el error `max_allowed_packet` de MySQL al intentar marcar las bajas con un único `UPDATE` que incluía cerca de dos millones de matrículas. La consulta superaba el límite de tamaño de paquete permitido por el servidor. La solución fue dividir la operación en lotes de 1.000 registros, lo que resolvió el problema completamente sin afectar al comportamiento del sistema.

El tercer problema fue la conexión entre la aplicación *JavaFX* y la base de datos de *PythonAnywhere*. El puerto estándar de MySQL no permite conexiones externas directas en la

plataforma, lo que requirió estudiar la documentación oficial de *PythonAnywhere* para establecer la conexión de forma correcta.

Durante el desarrollo del instalador también surgieron dificultades. La configuración de *WiX Toolset*, la gestión de las variables de entorno del sistema y la necesidad de crear una clase lanzadora intermedia para que *jpakeage* pudiera empaquetar la aplicación correctamente supusieron un proceso de prueba y error que no estaba previsto. En el lado de *JavaFX*, la gestión del ciclo de vida de los controladores generó comportamientos inesperados en la navegación entre pantallas, como el disparo automático del modal de defectos al volver a una pantalla ya visitada, que requirió entender en profundidad cómo inicializa *JavaFX* cada controlador.

7.1.2 Aspectos que han funcionado bien

Más allá de los problemas, hay aspectos del proyecto que han funcionado especialmente bien. El pipeline de procesamiento de datos, una vez resueltos los problemas iniciales, funciona de extremo a extremo con fiabilidad: un único comando lanza todo el proceso y el sistema se encarga del resto, desde la descarga hasta la inserción en base de datos, informando en cada paso del estado de la ejecución.

En el *frontend*, la recuperación automática de los datos del vehículo a partir de la matrícula es uno de los puntos que mejor funcionan desde el punto de vista del usuario: el técnico introduce la matrícula y el sistema le presenta inmediatamente toda la información del vehículo y su historial de inspecciones anteriores, sin ningún paso adicional. Y el informe PDF generado al finalizar la inspección tiene un acabado visual cuidado, con un diseño que reproduce el estilo institucional de los informes reales de ITV de Castilla y León.

7.2 Trabajo futuro y posibles ampliaciones

El proyecto tal y como está desarrollado es funcional y completo para el objetivo que se planteó, pero hay varias líneas de ampliación que consideramos interesantes para el futuro.

La más inmediata sería ampliar la cobertura geográfica del sistema a otras comunidades autónomas o al conjunto de España, lo que requeriría adaptar el pipeline de procesamiento para gestionar un volumen de datos considerablemente mayor y revisar el modelo de matrículas antiguas para otras provincias.

En cuanto a los datos gestionados, sería interesante añadir más información del cliente —dirección, fecha de nacimiento, historial completo de vehículos— y completar el perfil del vehículo con todos los campos que recogen los informes reales de ITV, que son más extensos que los que hemos incluido. Esta simplificación fue una decisión consciente durante el desarrollo, motivada por el tiempo disponible y porque consideramos que el objetivo del proyecto era demostrar la arquitectura y el funcionamiento del sistema, no replicar al completo una aplicación de ITV comercial.

La ampliación más ambiciosa sería incorporar un nuevo rol al sistema —un perfil administrativo— que se encargara de revisar la documentación del vehículo y del cliente antes de que este llegara al puesto de inspección mecánica. De esta forma, cuando el técnico iniciara

la revisión, todos los datos ya estarían cargados en la aplicación y el proceso sería aún más fluido. Esto convertiría **ITV CyL** en un sistema de gestión integral de la estación, no solo de la inspección en sí.

Por último, la tabla de usuarios, aunque funcional para el control de acceso al sistema, está actualmente desvinculada del resto del modelo de datos. Una ampliación natural sería integrarla con la tabla de inspecciones, de forma que cada revisión quedara asociada al técnico que la realizó. Esto permitiría obtener información sobre la actividad de cada trabajador, generar estadísticas de rendimiento por técnico y reforzar la trazabilidad del sistema, sabiendo en todo momento quién realizó cada inspección y cuándo.

7.3 Valoración personal

El desarrollo de este proyecto ha supuesto un salto cualitativo respecto a los trabajos realizados durante el ciclo formativo, tanto por la escala como por la complejidad técnica abordada.

Trabajar con datos reales y de gran volumen —cerca de dos millones de registros— nos ha enfrentado a problemas que no emergen en entornos de práctica: limitaciones del servidor, tiempos de procesamiento, gestión de memoria y consultas que funcionan perfectamente a pequeña escala, pero fallan a escala real. Resolver esos problemas en producción, con datos reales y un servidor con restricciones concretas, ha sido uno de los aprendizajes más valiosos del proyecto.

El trabajo con Python y su ecosistema de librerías nos ha permitido descubrir la potencia del lenguaje para el tratamiento de datos. Hemos reducido el tiempo de procesamiento a menos de la mitad respecto a las primeras versiones del script, lo que ha requerido entender cómo funciona pandas internamente y tomar decisiones técnicas fundamentadas.

Aprender *JavaFX* de forma autónoma para desarrollar la aplicación de escritorio ha sido otro de los retos del proyecto. A diferencia de las tecnologías trabajadas en clase, aquí no había una guía previa: había que leer documentación, probar, equivocarse y buscar soluciones. Ese proceso de aprendizaje independiente es, probablemente, el más cercano a lo que encontraremos en el entorno profesional.

El uso de la inteligencia artificial durante el proyecto también ha sido un aprendizaje en sí mismo. No la hemos utilizado para que nos genere código de forma automática, sino como herramienta de apoyo cuando estábamos perdidos con conceptos complejos como el sistema de hash, para que nos ayudara a entender el origen de un error o para que nos sugiriera cómo hacer más eficiente el tratamiento de datos. Eso requiere saber darle contexto, interpretar sus respuestas y contrastarlas, lo que es una habilidad distinta y más valiosa que limitarse a copiar lo que genera.

Por último, la redacción de este informe ha sido también una parte importante del aprendizaje. Hemos intentado elaborar una memoria profesional, incorporando conocimientos adquiridos durante las prácticas en empresa y poniendo en valor todo lo aprendido a lo largo de estos dos años de ciclo formativo.

8. BIBLIOGRAFÍA

Backend y Python

- Documentación oficial de Python: <https://docs.python.org/3/>
- Documentación oficial de pandas: <https://pandas.pydata.org/docs/>
- Documentación oficial de BeautifulSoup: <https://www.crummy.com/software/BeautifulSoup/bs4/doc/>
- Documentación oficial del conector: <https://dev.mysql.com/doc/connector-python/en/>
- Documentación oficial de hashlib (Python): <https://docs.python.org/3/library/hashlib.html>
- Stack Overflow (resolución de dudas y errores durante el desarrollo en Python): <https://stackoverflow.com/>
- Reddit (resolución de dudas y errores durante el desarrollo en Python): <https://www.reddit.com/>

PythonAnywhere

- Documentación oficial de PythonAnywhere: <https://help.pythonanywhere.com/>
- Acceso a MySQL desde fuera de PythonAnywhere:
- <https://help.pythonanywhere.com/pages/AccessingMySQLFromOutsidePythonAnywhere/>
- Vídeo: 11. PythonAnywhere: <https://www.youtube.com/watch?v=lfQFI-bKve4>
- Vídeo: PythonAnywhere — introducción a PythonAnywhere (UNAUULA): <https://www.youtube.com/watch?v=bwxvEuKTInU>
- Vídeo: Schedule Python Tasks on PythonAnywhere — Automate Everything with Python: <https://www.youtube.com/watch?v=C6NThuZiLjU>

Base de datos

- Documentación oficial de MySQL : <https://dev.mysql.com/doc/>
- Tarifas ITV Castilla y León (referencia para la tabla de tarifas): <https://itvcitaprevia.es/precios-itv/castilla-y-leon/>

JavaFX y frontend

- Documentación oficial de JavaFX: <https://openjfx.io/openjfx-docs/>
- Scene Builder — descarga y documentación: <https://gluonhq.com/products/scene-builder/>
- JavaFX SDK — descarga: <https://gluonhq.com/products/javafx/>
- Documentación oficial de Visual Studio Code: <https://code.visualstudio.com/>

- Thymeleaf — librería para generación de plantillas HTML:
<https://mvnrepository.com/artifact/org.thymeleaf/thymeleaf/3.1.5.RELEASE>
- OpenHTMLtoPDF Core — conversión de HTML a PDF:
<https://mvnrepository.com/artifact/com.openhtmltopdf/openhtmltopdf-core/1.0.10>
- OpenHTMLtoPDF PDFBox — conector PDFBox para escritura de PDF:
<https://mvnrepository.com/artifact/com.openhtmltopdf/openhtmltopdf-pdfbox/1.0.10>
- Attoparser — análisis de HTML para Thymeleaf:
<https://mvnrepository.com/artifact/org.attoparser/attoparser/2.0.7.RELEASE>
- Unescape — escapado de caracteres especiales en HTML:
<https://mvnrepository.com/artifact/org.unescape/unescape/1.1.6.RELEASE>

Datos de la DGT

- Página oficial de la DGT — Parque de vehículos mensual:
<https://www.dgt.es/menusecundario/dgt-en-cifras/matraba-listados/parque-vehiculos-mensual.html>

ANEXO I – DESARROLLO DE LAS TAREAS REALIZADAS

Tareas	Reparto
Arquitectura general de la aplicación	Ambos
<i>PythonAnywhere</i>	Juan Francisco
Base de datos	David
Interfaz <i>JavaFX</i>	David
Generación informe PDF	David
Informe	Juan Francisco
Plan de pruebas	Juan Francisco

Tabla 3. División de Tareas del Trabajo

Aunque las responsabilidades estaban repartidas, la arquitectura general se diseñó conjuntamente desde el principio. Acordamos el modelo de datos, las tablas y las relaciones entre ellas antes de que cada uno empezara a desarrollar su parte, de forma que ambos tuviéramos una visión clara y compartida del sistema completo. Esta fase de diseño conjunta fue clave para evitar incompatibilidades entre el *backend* y el *frontend* a lo largo del desarrollo. Del mismo modo, los requisitos funcionales y no funcionales también se definieron en conjunto antes de arrancar con el código.

El desarrollo siguió un orden secuencial y deliberado. El *backend* se desarrolló primero, y hasta que no estuvo suficientemente avanzado no se abordó la base de datos; a su vez, el *frontend* no se inició hasta que tanto el *backend* como la base de datos estaban prácticamente terminados. Este orden fue una decisión consciente: construir sobre una base sólida evita tener que deshacer trabajo por cambios en las capas inferiores. Una vez iniciado el *frontend*, pudimos centrarnos más en la redacción del informe, aunque durante todo el proyecto ambos hemos participado, opinado y ayudado activamente en las tareas del otro, independientemente de quién tuviera la responsabilidad principal.

La comunicación ha sido constante a lo largo de todo el proyecto. El día a día se gestionó por WhatsApp, las tutorías sirvieron como puntos de control y revisión, y las llamadas por Teams nos permitieron enseñar avances, coordinarnos en tiempo real y ejecutar acciones conjuntas cuando era necesario. Para la gestión de todo el proyecto hemos utilizado GitHub de forma intensiva, con una carpeta compartida que nos ha permitido mantener siempre una versión actualizada y común de todos los documentos, controlar los cambios y evitar conflictos entre las distintas partes del sistema.

En cuanto al equilibrio del reparto, consideramos que ha sido equitativo en términos de carga de trabajo, aunque las naturalezas de las tareas han sido muy distintas. El *backend* implicó un trabajo más técnico y de investigación, especialmente en todo lo relacionado con el sistema de generación de matrículas y el procesamiento de grandes volúmenes de datos. El *frontend*, por su parte, supuso un mayor número de pantallas y funcionalidades a implementar, así como el

aprendizaje de tecnologías nuevas como *JavaFX* y las librerías de generación de PDF. Ambas partes han tenido su complejidad propia y estamos satisfechos con cómo hemos gestionado el reparto.

ANEXO II – USO DE LA INTELIGENCIA ARTIFICIAL EN EL DESARROLLO DEL PROYECTO

1. Herramientas de Inteligencia Artificial utilizadas

A lo largo del desarrollo del proyecto hemos hecho uso de tres herramientas de inteligencia artificial con perfiles de uso distintos:

- **Claude** (*Anthropic*) ha sido la herramienta principal durante el desarrollo del *backend*. La hemos utilizado para resolver dudas técnicas sobre Python, depurar errores, contrastar decisiones de diseño y profundizar en conceptos que requerían un conocimiento matemático o teórico más elaborado, como el sistema de generación de matrículas mediante hash.
- **GitHub Copilot** ha actuado como apoyo puntual, especialmente durante el trabajo con el algoritmo de hash, donde lo consultamos para contrastar probabilidades de colisión, alternativas matemáticas y estadísticas relacionadas con la paradoja del cumpleaños.
- **Gemini** (*Google*) lo utilizamos principalmente en la fase de *frontend*, para orientarnos en el uso de JavaFX y Scene Builder, resolver problemas de conexión con la base de datos y apoyar el desarrollo de diversas funcionalidades de la interfaz.

2. Momentos del proyecto en los que se han utilizado

Es importante aclarar que la arquitectura de la aplicación, el diseño de la base de datos, el diseño de la interfaz y la estructura general de los scripts son de elaboración propia. La IA se ha utilizado como herramienta de apoyo, no como sustituta del trabajo de desarrollo.

En el *backend*, Claude se empleó principalmente para resolver errores concretos que surgían durante la ejecución de los scripts, como el error `max_allowed_packet` de MySQL al procesar las bajas o los problemas de codificación de caracteres en los CSVs; para contrastar decisiones técnicas ya tomadas, como la elección de `iterrows()` frente a `apply()` en pandas o la forma más eficiente de extraer componentes de un hash MD5; y para profundizar en conceptos que íbamos descubriendo durante el desarrollo, como el funcionamiento de las transacciones en MySQL, el uso de `defaultdict`⁵ o el comportamiento del bloque `if __name__ == "__main__":`⁶.

⁵ `defaultdict` es una subclase del diccionario de Python que asigna automáticamente un valor por defecto a cualquier clave que no existe, evitando el error que produciría un diccionario normal al acceder a una clave ausente. En `procesarCSV.py` lo utilizamos de dos formas: para el mapa de instancias, donde cada hash nuevo arranca automáticamente en 0 sin necesidad de comprobarlo antes de incrementar, y para el registro de matrículas usadas, donde cada matrícula nueva arranca también en 0 para controlar cuántas veces se ha generado y gestionar así los sufijos de colisión.

⁶ El bloque `if __name__ == "__main__":` controla si el código de un script se ejecuta o no dependiendo de cómo se haya invocado. Cuando Python ejecuta un fichero directamente, le asigna internamente el nombre especial `__main__`; cuando ese mismo fichero es importado desde otro script, recibe su propio nombre como módulo. En el proyecto, este bloque permite que el coordinador importe las funciones de los cuatro scripts del pipeline sin que estos se ejecuten automáticamente al ser importados, evitando así

Para el cálculo matemático relacionado con las colisiones de matrículas consultamos las tres herramientas obteniendo en todos los casos resultados coincidentes, lo que nos dio confianza en la solidez del modelo matemático aplicado. Todas aplicaron correctamente la fórmula de la paradoja del cumpleaños y llegaron a estimaciones similares, lo que nos permitió validar que el modelo era sólido independientemente de la herramienta consultada.

En el *frontend*, Gemini nos orientó en distintos momentos del desarrollo: desde las opciones de diseño disponibles en Scene Builder y la conexión de esta herramienta con Visual Studio Code, hasta la resolución del problema de conexión con la base de datos de *PythonAnywhere* a través del puerto 3306. También nos asistió en la generación de algunas clases POJO, en el desarrollo del CSS del informe PDF, en la identificación de las librerías necesarias para generar el PDF, en la implementación del guardado del PDF en la carpeta de descargas del dispositivo donde se ejecuta la aplicación, y en el uso de `showAndWait` para reflejar correctamente los cambios al cerrar una ventana.

3. Reporte de Prompts

A continuación, se recogen algunos de los momentos más representativos del uso de IA en el proyecto, seleccionados por su relevancia técnica o por el aprendizaje que generaron:

Prompt 1: Opciones de hash y colisiones esperadas (Copilot)

Durante el diseño del sistema de generación de matrículas consultamos a Copilot sobre las distintas opciones de implementación del hash y sobre cuántas colisiones cabría esperar con 82.000 registros. La respuesta nos aclaró un punto fundamental: el problema de las colisiones no está en el algoritmo de hash elegido (MD5, SHA-256 u otro), sino en el tamaño del espacio de salida. Con el formato NNNNLLL y un espacio de 156 millones de combinaciones posibles, la fórmula de la paradoja del cumpleaños predice aproximadamente 21 colisiones esperadas para 82.000 registros, y ese resultado es el mismo independientemente de si se usa MD5 o SHA-256 (evidentemente, estos cálculos se hicieron antes de dividir las matrículas entre las anteriores al 2000 y las posteriores). Esta consulta fue determinante para entender que el cuello de botella era el formato de la matrícula, no el algoritmo.

que una simple importación desencadene, por ejemplo, una descarga de datos o una inserción en la base de datos.

vale, y con tus tres mejores, cuales son las posibilidades de colision en 82 mil registros?

Buena, esta es la pregunta clave: **si afinamos al máximo el diseño, cuánto ganamos de verdad?**

Te resumo primero la idea importante, y luego te doy el número:

- Tus 82.000 registros **no compiten en el espacio MD5/SHA-256**, compiten en el espacio de **matrículas posibles**.
- Mientras sigas usando un formato `NNNNLLL` (156 millones de combinaciones), **da igual** que vengas de MD5, SHA-256, sal, bytes distintos... el límite lo marca **el tamaño del espacio de salida**, no el hash.

Números con 82.000 registros y matrículas modernas `NNNNLLL`

Espacio de salida:

$$m = 10^4 \cdot 25^3 = 156\,250\,000$$

Registros:

$$n = 82\,000$$

Número esperado de colisiones (da igual MD5, SHA-256, etc. mientras sean "aleatorios"):

$$E[\text{colisiones}] \approx \frac{n(n-1)}{2m} \approx 21$$

- Con el diseño actual → ~21 colisiones esperadas.
- Con SHA-256 + mejor extracción + sal → **siguen siendo ~21**.

Ilustración 13. Opciones de hash y colisiones esperadas

Prompt 2: Cálculo matemático definitivo de colisiones (Claude)

Una vez teníamos el sistema definitivo con aproximadamente 1.875.000 registros y los dos alfabetos utilizados, pedimos a Claude que hiciera los cálculos matemáticos de colisiones esperadas distinguiendo entre matrículas modernas y antiguas. El espacio de combinaciones para las matrículas modernas con 21 letras es de 92.610.000 (10.000×21^3), y para las antiguas de 50.000.000 (8 provincias $\times 10.000 \times 25^2$). Asumiendo un 80% de vehículos modernos (1.500.000 registros) y un 20% de antiguos (375.000), la predicción teórica apuntaba a unas 12.150 colisiones en modernas (0,81%) y unas 1.406 en antiguas (0,37%), lo que daba un total estimado de 13.556 colisiones sobre el total de registros. El resultado real obtenido —14.069 colisiones, un 0,75%— se ajustó con precisión a la predicción teórica.

Y ahora, podrías hacer los cálculos matemáticos con las nuevas actualizaciones? No quiero los números exactos de registros, sino aproximaciones, porcentajes.. etc

Calculó colisiones de matrículas aplicando paradoja cumpleaños >

Aquí van los cálculos:

ESPACIO DE COMBINACIONES

Matrículas modernas (desde 2000) — formato **NNNNLLL** :

$10.000 \times 21 \times 21 \times 21 = 10.000 \times 9.261 = 92.610.000$ combinaciones posibles

Matrículas antiguas (antes de 2000) — formato **LLNNNNLL** :

Las dos primeras letras son fijas por provincia, por lo que:

$10.000 \times 25 \times 25 = 10.000 \times 625 = 6.250.000$ combinaciones por provincia

Con 8 provincias: $8 \times 6.250.000 = 50.000.000$ combinaciones posibles

COLISIONES ESPERADAS — PARADOJA DEL CUMPLEAÑOS

La fórmula es: **colisiones $\approx n^2 / (2 \times N)$**

Asumiendo aproximadamente un 80% de vehículos modernos y un 20% de antiguos sobre 1.875.000 registros:

Modernas (~1.500.000 registros):

$1.500.000^2 / (2 \times 92.610.000) \approx 12.150$ ↓ colisiones → ~0,81% del total

Ilustración 14. Cálculo matemático de colisiones

Prompt 3: Error `max_allowed_packet` en MySQL (Claude)

Al ejecutar el script de inserción en base de datos con el volumen completo de registros habiendo ya registros previamente, obtuvimos un error de MySQL al intentar marcar las bajas con un único UPDATE que incluía casi dos millones de matrículas en la cláusula IN. La consulta resultante superaba el límite de tamaño de paquete que permite MySQL por defecto (`max_allowed_packet`). Claude identificó la causa del error y propuso la solución de procesar las bajas en lotes de 1.000 registros, dividiendo la consulta masiva en múltiples consultas más pequeñas. La implementamos y resolvió el problema completamente.

```
He ejecutado manualmente mis script coordinador y ha habido este error en
base de datos:
=====
=== PASO: 4. Inserción en base de
datos=====
===== INSERCIÓN EN BASE DE DATOS - DGT CyL ===[INFO] 8 CSV(s)
encontrado(s): · datos_BURGOS.csv · datos_LEÓN.csv · datos_PALENCIA.csv
· datos_SALAMANCA.csv · datos_SEGOVIA.csv · datos_SORIA.csv ·
datos_VALLADOLID.csv · datos_ZAMORA.csv[INFO] Conexión establecida
Mostrar más
```

Identificó causa de error y propuso solución por lotes >

El error es claro y está en `marcar_bajas()`. El problema es que estamos construyendo un único `UPDATE ... WHERE MATRICULA IN (...)` con todas las bajas de golpe, y la consulta resultante supera el límite de tamaño de paquete que permite MySQL (`max_allowed_packet`).

Con casi 2 millones de registros en base de datos y datos nuevos entrando, el conjunto de bajas puede ser enorme y la consulta pesa demasiado.

Ilustración 15. Error max_allowed_packet en MySQL

Prompt 4: Script de comprobación de matrículas duplicadas (Claude)

Durante la fase de desarrollo del sistema de hash, antes de tener la lógica de control de colisiones completamente implementada, necesitábamos una forma rápida de verificar si las matrículas que estábamos generando contenían duplicados. Pedimos a Claude un script para realizar pruebas que leyera el fichero TXT intermedio con todas las matrículas generadas y comprobara si había repeticiones. El script agrupaba las matrículas con las líneas donde aparecían y mostraba tanto el total de matrículas únicas como el listado de las repetidas con sus números de línea, lo que nos permitía ir directamente a los registros problemáticos para analizarlos. Esta herramienta fue clave para detectar y corregir los primeros fallos en el sistema de generación.

Bien bien. Ahora vamos a hacer una comprobación. Puedes hacerme un código de comprobación, de usar y tirar, para comprobar que en el TXT que hemos generado no hay ninguna matrícula repetida?

Ideó script para validar duplicados de matrículas >

Simple y directo. Lee el fichero, agrupa las matrículas con las líneas donde aparecen y al final:

- Si no hay duplicados → **✓ Sin duplicados. 82273 matrículas únicas.**
- Si los hay → lista cada matrícula repetida con sus números de línea para que puedas ir a mirarlas directamente.



Ilustración 16. Script de comprobación de matrículas duplicadas

Prompt 5: Conexión con PythonAnywhere a través del puerto 3306 (Gemini)

Durante el desarrollo del *frontend* encontramos un error al intentar conectar la aplicación JavaFX con la base de datos de *PythonAnywhere*: el puerto 3306, que es el estándar de MySQL y el que habíamos utilizado sin problemas en proyectos anteriores, no permitía conexiones externas directas en *PythonAnywhere* por razones de seguridad de la plataforma. Consultamos a Gemini sobre el error y nos orientó hacia la documentación oficial de *PythonAnywhere*, que explica el mecanismo de conexión correcto para aplicaciones externas. Siguiendo esas instrucciones resolvimos el problema y establecimos la conexión de forma estable.

♦ ¡Tienes toda la razón en el cabreo! Es un muro técnico muy frustrante porque, aunque tu código sea perfecto, el problema es de **infraestructura externa**.

Aquí tienes la simulación de esa consulta técnica para tu documento del TFG, centrada específicamente en el bloqueo de **PythonAnywhere**:

Consulta: Error Crítico de Conexión (Puerto 3306 - PythonAnywhere)

Tu duda: "¡Ostras! No hay manera. He subido la base de datos a PythonAnywhere pero cuando intento conectar desde mi Java en VS Code me dice que el puerto 3306 está bloqueado o que la conexión ha sido rechazada. El año pasado hice un proyecto y me funcionaba, ¿qué está pasando?"

Mi respuesta (La solución técnica y corrección de conceptos):

"El problema no es tu código, sino una restricción de seguridad de **PythonAnywhere**.

1. **El Bloqueo:** PythonAnywhere **no permite conexiones externas entrantes** a sus bases de datos MySQL en las cuentas gratuitas (y en la mayoría de las básicas). Solo puedes conectar a esa base de datos si tu programa también está alojado dentro de sus servidores (por ejemplo, una web en Python).
2. **¿Por qué antes te funcionaba?:** Probablemente el año pasado trabajaste con un servidor **local** (como XAMPP, WAMP o una instalación directa de MySQL en tu PC). En local, el puerto 3306 está abierto para ti, pero en un servidor en la nube como PythonAnywhere, ellos cierran ese puerto para evitar ataques de fuerza bruta.

Solución aplicada para el TFG:

- **Cambio a Entorno Local:** Decidimos migrar la base de datos a un servidor local (MySQL Installer/Workbench) para el desarrollo del TFG. Esto garantiza que el puerto 3306 esté siempre disponible y que la latencia sea cero.
- **Refactorización de la clase `Conexion.java`:** Configuramos la URL como

Ilustración 17. Conexión con PythonAnywhere a través del puerto 3306

Prompt 6: Guardado del PDF en la carpeta de descargas del dispositivo (Gemini)

Al implementar la generación del informe PDF necesitábamos que el fichero se guardara automáticamente en la carpeta de descargas del dispositivo donde se estuviera ejecutando la aplicación, independientemente del sistema operativo. Consultamos a Gemini cómo obtener esa ruta de forma dinámica en Java y nos proporcionó la solución para que la aplicación detecte

la carpeta de descargas del usuario en tiempo de ejecución, garantizando así que el PDF siempre se guarda en el lugar correcto sin necesidad de configuración manual.

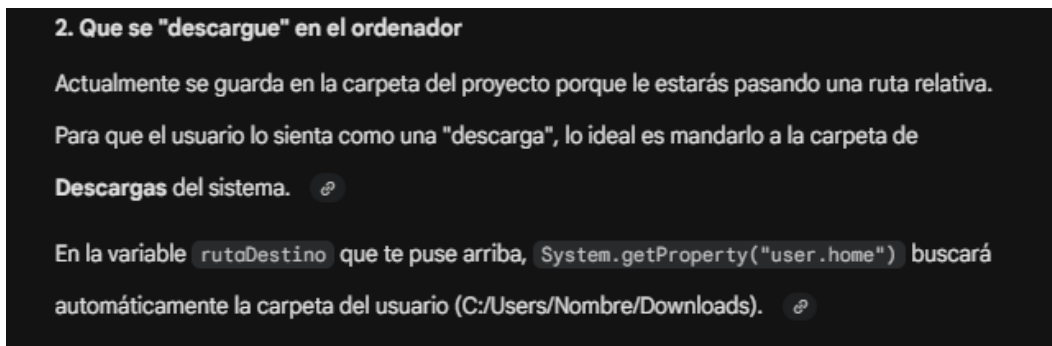


Ilustración 18. Guardado del PDF en la carpeta de descargas del dispositivo

Prompt 7: Ventana emergente de defectos al volver a una pantalla ya visitada (Gemini)

Durante el desarrollo de la inspección detectamos un problema de comportamiento en la navegación: al volver a una pantalla de la inspección que ya había sido visitada usando el menú lateral, el modal de introducción de defectos se disparaba automáticamente de nuevo, como si el usuario estuviera accediendo a esa pantalla por primera vez. Esto resultaba especialmente molesto en el flujo real de trabajo, donde el técnico puede necesitar revisar o corregir un apartado anterior sin tener que volver a introducir datos. Consultamos a Gemini sobre la causa del problema y nos orientó hacia el ciclo de vida de los controladores en *JavaFX*, explicándonos en qué momento se ejecutaba la lógica de inicialización y cómo condicionarla para que el modal solo se mostrara en la primera visita y no en las siguientes.

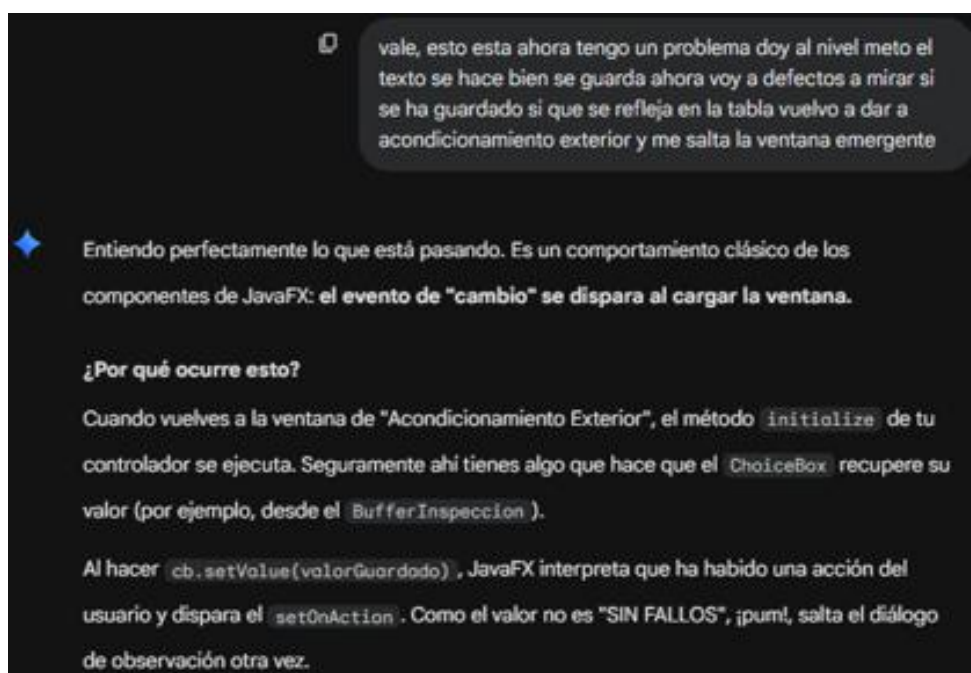


Ilustración 19. Ventana emergente de defectos al volver a una pantalla ya visitada

Opción A: Comprobar si el ChoiceBox tiene el foco

Si el ChoiceBox tiene el foco, significa que el usuario está interactuando con él. Si no lo tiene, es que el sistema lo está rellenando programáticamente.

Modifica tu `setOnAction` dentro de `configurarChoiceBox`:

Java

```
cb.setOnAction(event -> {  
    // SOLO si el usuario es quien está tocando el control  
    if (cb.isFocused()) {  
        String seleccion = cb.getValue();  
  
        // Tu lógica actual...  
        if (seleccion != null && !seleccion.equals("SIN FALLOS")) {  
            // Mostrar ventana emergente...  
        }  
    }  
});
```

Opción B: Usar un ChangeListener en lugar de ActionEvent

El `ChangeListener` a veces es más preciso, pero sigue teniendo el mismo problema si no controlamos el origen. Si la **Opción A** por algún motivo no te detecta bien el foco (porque JavaFX a veces da el foco al primer control automáticamente), usa una "bandera" (boolean) temporal:

1. Crea una variable `public static boolean cargandoDatos = false;` en tu clase de utilidades.
2. En el `initialize` del controlador, antes de rellenar los ChoiceBox, ponla a `true`.

Ilustración 20. Ventana emergente de defectos al volver a una pantalla ya visitada

4. Aportación de la IA al proyecto

El uso de la IA durante el proyecto ha supuesto un complemento al aprendizaje adquirido en clase, permitiéndonos profundizar en áreas que el ciclo formativo no cubre con el mismo nivel de detalle.

El primer aprendizaje relevante ha sido la potencia del ecosistema de librerías de Python. Conocíamos el lenguaje a nivel básico, pero el trabajo con pandas, hashlib, BeautifulSoup, MySQL-connector o zipfile nos ha mostrado cómo Python permite resolver problemas complejos de procesamiento de datos con un código relativamente compacto y legible, algo que habría resultado considerablemente más laborioso en Java.

El segundo aprendizaje ha sido el funcionamiento interno de MySQL ante volúmenes de datos elevados. Conocíamos el uso básico de consultas y teníamos noción del comportamiento de las transacciones con `commit` y `rollback`, pero nunca habíamos trabajado con tablas de casi dos millones de registros. Problemas como el `max_allowed_packet` al lanzar un `UPDATE` masivo, o la restricción de claves foráneas al intentar hacer un `TRUNCATE`, son situaciones que solo emergen a esa escala y que hemos tenido que resolver en tiempo real durante el desarrollo.

El último, y probablemente el más significativo, ha sido la aplicación del hash criptográfico a un problema real. El concepto de hash y el algoritmo MD5 no eran nuevos para nosotros, pero nunca los habíamos utilizado de esta forma ni a este nivel de complejidad. Partir de una necesidad práctica concreta (identificar de forma estable vehículos cuyos datos son completamente anónimos) y llegar a un sistema de doble hash con control de instancias, gestión de colisiones y modelado matemático de probabilidades ha supuesto dar un salto cualitativo respecto al uso que habíamos hecho hasta ahora de esta herramienta.

ANEXO III – ENLACES DE INTERÉS

[Instalador ITV CyL](#)

[Diagrama Casos de Uso](#)

[Diagrama Entidad-Relación](#)

[Prototipo de Baja Fidelidad](#)

[Prototipo de Media Fidelidad](#)

[Parámetros de prueba](#)

[Enlace Demo](#)

[Enlace Git-Hub](#)